

Package ‘Capr’

September 22, 2026

Title Cohort Definition Application Programming

Version 3.0.0

Description Provides a programming language for defining cohort definitions in R to use in studies for Observational Health Data Sciences and Informatics (OHDSI). The functions in 'Capr' allow for the programmatic creation of OHDSI concept sets and cohorts that can be serialized to 'OHDSI' compatible 'json' files or to 'OHDSI-SQL'. 'Capr' functions can be used to create, save, and load component parts to a cohort definition allowing R programmers to easily reuse cohort logic. 'Capr' provides tools to create a large number of OHDSI cohorts while also helping bridge the gap between human readable descriptions of clinical phenotypes and their computational implementation.

License Apache License (>= 2)

URL <https://ohdsi.github.io/Capr/>, <https://github.com/OHDSI/Capr/>

BugReports <https://github.com/OHDSI/Capr/issues/>

Encoding UTF-8

Depends R (>= 4.2.0)

Imports jsonlite,
methods,
purrr (>= 1.0.1),
rlang,
dplyr,
tidyr,
checkmate,
tibble,
readr,
stringr,
glue,
cli,
digest,
fs,
lubridate,
DBI,
DatabaseConnector,
SqlRender,

CirceR,
jsonvalidate

Suggests testthat (>= 3.0.0),
knitr,
rmarkdown,
Eunomia,
bit64,
PhenotypeLibrary (>= 3.36.0),
rstudioapi,
waldo

VignetteBuilder knitr

Config/testthat/edition 3

Collate 'Capr.R'
'conceptSet.R'
'attributes-concept.R'
'attributes-conceptSetSelection.R'
'attributes-dateAdjustment.R'
'attributes-logic.R'
'query.R'
'window.R'
'criteria.R'
'attributes-nested.R'
'attributes-op.R'
'exit.R'
'cohort.R'
'collectCodesetId.R'
'conceptSearch.R'
'jsonToCapr.R'
'print.R'

Remotes ohdsi/PhenotypeLibrary

Config/roxygen2/version 8.0.0

RoxygenNote 7.3.2

R topics documented:

==	6
age	7
ageAtEnd	7
ageAtStart	8
as.data.frame,ConceptSet-method	8
as.json	9
atLeast	9
atMost	10
attrition	11
bt	11
CensoringCriteria-class	12
censoringEvents	13
cohort	13
compile	14
compile,Cohort-method	14

Concept-class	15
conceptAttribute-class	15
ConceptSet-class	16
conceptSetAttribute-class	16
ConceptSetItem-class	16
conditionEra	17
conditionOccurrence	17
conditionSourceConcept	18
conditionStatus	18
conditionStatusCS	19
conditionType	19
conditionTypeCS	20
conditionTypeExclude	20
continuousObservation	21
Criteria-class	21
cs	22
dateAdjustment	23
dateAdjustmentAttribute-class	23
daysOfSupply	24
death	24
deathType	25
deathTypeCS	25
deathTypeExclude	26
detectUnsupportedKeys	26
deviceExposure	27
deviceType	27
deviceTypeCS	28
deviceTypeExclude	28
doseEra	29
doseUnit	29
doseUnitCS	30
doseValue	30
drugEra	31
drugExit	31
drugExposure	32
DrugExposureExit-class	33
drugQuantity	33
drugRefills	34
drugSourceConcept	34
drugType	35
drugTypeCS	35
drugTypeExclude	36
duringInterval	36
endDate	37
Endpoint-class	37
entry	37
eq	38
era	39
eraLength	40
ethnicityCS	40
EventAperture-class	41
eventEnds	41

eventStarts	42
EventWindow-class	42
exactly	43
exit	44
firstOccurrence	44
FixedDurationExit-class	45
fixedExit	45
genderConcepts	46
genderCS	46
getConceptDescendants	47
getConceptInfo	48
getConceptSetCall	48
getConceptSetDetails	49
Group-class	50
gt	50
gte	51
insertCaprCode	52
jsonToCapr	52
jsonToCaprFile	53
keyValueAttribute-class	54
logicAttribute-class	54
lotNumber	54
lt	55
lte	55
makeCohortSet	56
male	57
mapSourceToStandard	58
measurement	59
measurementOperator	59
measurementOperatorCS	60
measurementSourceConcept	60
measurementType	61
measurementTypeCS	61
measurementTypeExclude	62
measurementUnit	62
nbt	63
nestedAttribute-class	63
nestedWithAll	64
nestedWithAny	64
nestedWithAtLeast	64
nestedWithAtMost	65
observation	65
observationExit	66
ObservationExit-class	66
observationPeriod	66
observationPeriodType	67
observationQualifier	67
observationQualifierCS	68
observationSourceConcept	68
observationType	69
observationTypeCS	69
observationTypeExclude	70

ObservationWindow-class	70
Occurrence-class	70
occurrenceCount	71
opAttributeCharacter-class	71
opAttributeDate-class	71
opAttributeInteger-class	72
opAttributeNumeric-class	72
opAttributeSuper-class	72
payerConcept	73
payerPlanPeriod	73
payerSourceConcept	74
periodLength	74
periodTypeCS	75
placeOfService	75
placeOfServiceCS	76
planConcept	76
planSourceConcept	77
procedure	77
procedureModifier	78
procedureModifierCS	78
procedureSourceConcept	79
procedureType	79
procedureTypeCS	80
procedureTypeExclude	80
providerSpecialtyConcepts	81
providerSpecialtyCS	81
quantityValue	82
Query-class	82
raceCS	82
rangeHigh	83
rangeHighRatio	83
rangeLow	84
rankedSearchConcepts	84
readConceptSet	85
routeConcept	86
routeConceptCS	86
searchConcepts	87
specimen	88
specimenAnatomicSite	88
specimenAnatomicSiteCS	89
specimenDiseaseStatus	89
specimenDiseaseStatusCS	90
specimenSourceId	90
specimenType	91
specimenTypeCS	91
specimenTypeExclude	92
sponsorConcept	92
sponsorSourceConcept	93
startDate	93
stopReason	94
stopReasonConcept	94
stopReasonSourceConcept	95

stringContains	95
stringEndsWith	96
stringNotContains	96
stringNotEndsWith	97
stringNotStartsWith	97
stringStartsWith	98
summary	98
toCirce	99
toCohortJson	99
toConceptSetJson	100
uniqueDeviceId	100
unitCS	101
valueAsConcept	101
valueAsConceptSet	102
valueAsNumber	102
valueAsString	103
visit	103
visitDetail	104
visitDetailSourceConcept	104
visitDetailTypeCS	105
visitLength	105
visitSourceConcept	106
visitType	106
visitTypeCS	107
visitTypeExclude	107
visitTypeSet	108
withAll	108
withAny	109
withAtLeast	109
withAtMost	110
writeCohort	110
writeConceptSet	111

Index	112
--------------	------------

==	<i>FUnction checks if two concept set class objects are equivalent</i>
----	--

Description

FUnction checks if two concept set class objects are equivalent

Usage

```
## S4 method for signature 'ConceptSet,ConceptSet'
e1 == e2
```

Arguments

e1, e2 a ConceptSet Class object

age	<i>Age attribute for a query</i>
-----	----------------------------------

Description

Filters criteria by the person's age at the event date. Pass any comparison operator ([lt](#), [gt](#), [gte](#), [lte](#), [eq](#), [bt](#), [nbt](#)).

Usage

```
age(op)
```

Arguments

op	An opAttribute built with lt() , gt() , bt() , etc.
----	---

Value

An integer opAttribute with name = "Age" for use in any domain query.

See Also

[ageAtStart](#), [ageAtEnd](#), [lt](#), [gt](#), [gte](#), [bt](#)

Examples

```
# Age 18 or older at index
conditionOccurrence(cs(1L, name = "test"), age(gte(18L)))
# Age between 18 and 65
conditionOccurrence(cs(1L, name = "test"), age(bt(18L, 65L)))
```

ageAtEnd	<i>Age at era end attribute (for era and period domains)</i>
----------	--

Description

Age at era end attribute (for era and period domains)

Usage

```
ageAtEnd(op)
```

Arguments

op	an opAttribute object (numeric or integer) defining the age comparison
----	--

Value

An attribute for use in [conditionEra\(\)](#), [drugEra\(\)](#), [doseEra\(\)](#), [observationPeriod\(\)](#).

ageAtStart	<i>Age at era start attribute (for era and period domains)</i>
------------	--

Description

Age at era start attribute (for era and period domains)

Usage

```
ageAtStart(op)
```

Arguments

op an opAttribute object (numeric or integer) defining the age comparison

Value

An attribute for use in conditionEra(), drugEra(), doseEra(), observationPeriod().

as.data.frame, ConceptSet-method	<i>Coerce a concept set expression to a dataframe</i>
----------------------------------	---

Description

Coerce a concept set expression to a dataframe

Usage

```
## S4 method for signature 'ConceptSet'
as.data.frame(x)
```

Arguments

x A Caper Concept Set

Value

A tibble (dataframe) with columns: concept_id, includeDescendants, isExcluded, includeMapped.

as.json

*Coerce Capr object to json***Description**

Coerce Capr object to json

Usage

```
as.json(x, pretty = TRUE, ...)
```

```
## S4 method for signature 'ConceptSet'
as.json(x, pretty = TRUE, ...)
```

```
## S4 method for signature 'Cohort'
as.json(x, pretty = TRUE, ...)
```

Arguments

x	the capr object
pretty	a toggle to make the json look nice, part of jsonlite
...	additional arguments passes to jsonlite::toJSON

atLeast

*Require at least N occurrences of a query within a window***Description**

Require at least N occurrences of a query within a window

Usage

```
atLeast(
  x,
  query,
  aperture = duringInterval(eventStarts(-Inf, Inf)),
  distinct = NA,
  countColumn = NA_character_
)
```

Arguments

x	Minimum integer count of required occurrences.
query	A Query object (e.g. from conditionOccurrence()).
aperture	An EventAperture built with duringInterval . Defaults to all time.
distinct	If TRUE, count distinct values of countColumn.
countColumn	Column to use for distinct counting (e.g. "DOMAIN_CONCEPT").

Value

A Criteria object for use in [withAll](#), [withAny](#), etc.

See Also

[exactly](#), [atMost](#), [duringInterval](#)

Examples

```
metformin <- cs(descendants(1503297L), name = "Metformin")
# At least 2 metformin fills in the 365 days before index
atLeast(2, drugExposure(metformin), duringInterval(eventStarts(-365, 0)))
```

atMost

Require at most N occurrences of a query within a window

Description

Require at most N occurrences of a query within a window

Usage

```
atMost(
  x,
  query,
  aperture = duringInterval(eventStarts(-Inf, Inf)),
  distinct = NA,
  countColumn = NA_character_
)
```

Arguments

x	Maximum integer count of allowed occurrences.
query	A Query object (e.g. from conditionOccurrence()).
aperture	An EventAperture built with duringInterval . Defaults to all time.
distinct	If TRUE, count distinct values of countColumn.
countColumn	Column to use for distinct counting (e.g. "DOMAIN_CONCEPT").

Value

A Criteria object for use in [withAll](#), [withAny](#), etc.

See Also

[exactly](#), [atLeast](#), [duringInterval](#)

Examples

```
t2dm <- cs(descendants(201826L), name = "T2DM")
# At most 1 T2DM diagnosis before index
atMost(1, conditionOccurrence(t2dm), duringInterval(eventStarts(-365, 0)))
```

attrition	<i>Define cohort inclusion/exclusion rules (attrition)</i>
-----------	--

Description

Each named argument is one attrition rule — a Group built with [withAll](#), [withAny](#), [withAtLeast](#), or [withAtMost](#). Rules are applied in order after the index event is selected.

Usage

```
attrition(..., expressionLimit = c("First", "All", "Last"))
```

Arguments

<code>...</code>	Named Group objects, each representing one inclusion or exclusion rule.
<code>expressionLimit</code>	Which qualifying events per person survive attrition — "First" (default), "All", or "Last". Should align with <code>primaryCriteriaLimit</code> in entry .

Value

A CohortAttrition object.

See Also

[cohort](#), [entry](#), [withAll](#), [withAny](#)

Examples

```
t2dm <- cs(descendants(201826L), name = "T2DM")
attrition(
  noT2DM = withAll(exactly(0, conditionOccurrence(t2dm),
    duringInterval(eventStarts(-Inf, 0)))),
  expressionLimit = "First"
)
```

bt	<i>Between operator</i>
----	-------------------------

Description

Builds a range comparison for use inside attribute functions such as [age](#), [valueAsNumber](#), [startDate](#), etc. Accepts integer, numeric, or Date inputs.

Usage

```

bt(x, y)

## S4 method for signature 'integer'
bt(x, y)

## S4 method for signature 'numeric'
bt(x, y)

## S4 method for signature 'Date'
bt(x, y)

```

Arguments

x	Lower bound (integer, numeric, or Date).
y	Upper bound; must match type of x.

Value

An opAttribute for use inside attribute constructor functions.

See Also

[nbt](#), [lt](#), [gt](#), [age](#)

Examples

```

# Age between 18 and 65
conditionOccurrence(cs(1L, name = "test"), age(bt(18L, 65L)))

```

CensoringCriteria-class

An S4 class identifying a censoring criteria for the cohort

Description

The censoring criteria specifies events where the person exits the cohort. These events are based on a query class object and users can specify multiple queries in the censoring criteria.

Slots

criteria a list of Capr query class objects that specify the events that would lead a person to exit the cohort.

censoringEvents	<i>Constructor for a set of censoring events</i>
-----------------	--

Description

Constructor for a set of censoring events

Usage

```
censoringEvents(...)
```

Arguments

... a list of Capr query objects that are used as censoring events

Value

a censoring criteria S4 object used to define the censoring events of the cohort definition

cohort	<i>Build a cohort definition</i>
--------	----------------------------------

Description

Assembles the four components of an OHDSI cohort definition into a single Cohort object that can be serialized to Circe-compatible JSON via [toCohortJson](#) or written to disk with [writeCohort](#).

Usage

```
cohort(entry, attrition = NULL, exit = NULL, era = NULL)
```

Arguments

entry	A CohortEntry from entry() .
attrition	Optional CohortAttrition from attrition() .
exit	Optional CohortExit from exit() . Defaults to observation exit.
era	Optional CohortEra from era() . Defaults to no era collapse.

Value

A Cohort S4 object.

See Also

[entry](#), [attrition](#), [exit](#), [era](#), [toCohortJson](#), [writeCohort](#)

Examples

```
giBleed <- cs(descendants(192671L), name = "GI Bleed")
ch <- cohort(
  entry = entry(conditionOccurrence(giBleed), primaryCriteriaLimit = "First"),
  exit  = exit(endStrategy = observationExit())
)
```

compile	<i>Compile a Capr object to json</i>
---------	--------------------------------------

Description

Compile a Capr object to json

Usage

```
compile(object, ...)
```

Arguments

object	A Capr object such as a cohort, list of cohorts, or concept set.
...	Arguments passed on to jsonlite::toJSON (e.g. pretty = TRUE). For the Cohort method, includeConceptSets is also allowed; see compile,Cohort-method .

Value

The json representation of the Capr object

compile,Cohort-method	<i>Compile a Capr cohort to json</i>
-----------------------	--------------------------------------

Description

Compile a Capr cohort to json

Compile a Capr Concept Set to json

Usage

```
## S4 method for signature 'Cohort'
compile(object, ..., includeConceptSets = NULL)

## S4 method for signature 'ConceptSet'
compile(object, ...)
```

Arguments

object	A Capr Concept Set created with ‘cs’
...	Arguments passed on to jsonlite::toJSON. e.g. ‘pretty = TRUE’ for nicely formatted json.
includeConceptSets	Optional list of ConceptSet objects to include in the JSON even if not referenced in the cohort (e.g. for round-trip equivalence).

Details

Deprecated; use [toCohortJson](#) instead.

Value

The json representation of Capr cohorts
 The json representation of Capr cohorts

Concept-class	<i>An S4 class for a single OMOP Concept</i>
---------------	--

Description

A concept class contains all the information about the concept from the OMOP vocabulary.

Slots

concept_id the OMOP/OHDSI concept ID
 concept_name the name of the concept
 standard_concept whether the concept is standard ‘S’, classification ‘C’, or non-standard NA
 standard_concept_caption Whether the concept is standard full phrase
 invalid_reason Whether the concept is invalid single letter
 invalid_reason_caption whether the concept is invalid standard phrase
 concept_code The original code of the concept from its vocabulary
 domain_id The domain of the concept (e.g. Drug, Condition, Procedure, etc)
 vocabulary_id the name of the vocabulary
 concept_class_id type of concept class

conceptAttribute-class	<i>An S4 class for a concept attribute</i>
------------------------	--

Description

An S4 class for a concept attribute

Slots

name the name of the attribute
 conceptSet a list representing the concepts for the attribute

ConceptSet-class	<i>An S4 class for ConceptSetExpression</i>
------------------	---

Description

A class for the concept set expressions bundles multiple concepts with mapping

Slots

id an id for the concept set expression to identify within a component

Name the name of the concept set expression

Expression a list containing expressions. expressions include multiple conceptSetItem objects

conceptSetAttribute-class	<i>An S4 class for a concept set attribute that holds a reference to a concept set ID</i>
---------------------------	---

Description

An S4 class for a concept set attribute that holds a reference to a concept set ID

Slots

name the name of the attribute

conceptSet a ConceptSet object that provides the ID reference

ConceptSetItem-class	<i>An S4 class for ConceptSetItem</i>
----------------------	---------------------------------------

Description

A class that provides information on the mapping of the concept

Slots

Concept a concept class object

isExcluded toggle if want to exclude the concept

includeDescendants toggle if want to include descendants

includeMapped toggle if want to include map

conditionEra	<i>Query the condition era domain</i>
--------------	---------------------------------------

Description

Query the condition era domain

Usage

```
conditionEra(conceptSet, ...)
```

Arguments

conceptSet	A ConceptSet for the condition.
...	Optional attributes: firstOccurrence(), startDate(), endDate(), eraLength(), occurrenceCount(), ageAtStart(), ageAtEnd(), etc.

Value

A Query object.

See Also

[conditionOccurrence](#), [entry](#)

conditionOccurrence	<i>Query the condition occurrence domain</i>
---------------------	--

Description

Query the condition occurrence domain

Usage

```
conditionOccurrence(conceptSet, ...)
```

Arguments

conceptSet	A ConceptSet built with cs() , or NULL for any condition.
...	Optional attributes: firstOccurrence(), startDate(), endDate(), age(), male()/female(), conditionType(), conditionStatus(), conditionSourceConcept(), nestedWithAll(), dateAdjustment(), etc.

Value

A Query object.

See Also

[entry](#), [atLeast](#), [cs](#)

Examples

```
t2dm <- cs(descendants(201826L), name = "T2DM")
conditionOccurrence(t2dm)
conditionOccurrence(t2dm, firstOccurrence(), age(gte(18L)))
```

conditionSourceConcept

Add a condition source concept attribute

Description

Add a condition source concept attribute

Usage

```
conditionSourceConcept(conceptSet)
```

Arguments

conceptSet a ConceptSet object containing the source concepts

Value

An attribute that can be used in a query function

conditionStatus

Add a condition status attribute

Description

Add a condition status attribute

Usage

```
conditionStatus(ids, connection = NULL, vocabularyDatabaseSchema = NULL)
```

Arguments

ids integer concept ids for the attribute

connection an optional connection to an OMOP CDM database, used only to look up concept names for display (e.g. in Atlas). When NULL (default) the attribute is built from the ids alone, which produces identical SQL.

vocabularyDatabaseSchema the schema of the OMOP vocabulary tables; only used when ‘connection’ is supplied

Value

An attribute for use in conditionOccurrence().

conditionStatusCS	<i>Condition status filter via concept set — ConditionOccurrence</i>
-------------------	--

Description

Condition status filter via concept set — ConditionOccurrence

Usage

```
conditionStatusCS(conceptSet, isExclusion = FALSE)
```

Arguments

conceptSet	A ConceptSet built with cs().
isExclusion	logical; if TRUE the types in the concept set are excluded.

Value

An attribute for use in conditionOccurrence().

conditionType	<i>Add a condition type attribute to determine the provenance of the record</i>
---------------	---

Description

Add a condition type attribute to determine the provenance of the record

Usage

```
conditionType(ids, connection = NULL, vocabularyDatabaseSchema = NULL)
```

Arguments

ids	integer concept ids for the attribute
connection	an optional connection to an OMOP CDM database, used only to look up concept names for display (e.g. in Atlas). When NULL (default) the attribute is built from the ids alone, which produces identical SQL.
vocabularyDatabaseSchema	the schema of the OMOP vocabulary tables; only used when 'connection' is supplied

Value

An attribute that can be used in a query function

conditionTypeCS	<i>ConditionType filter via concept set (ConceptSetSelection)</i>
-----------------	---

Description

ConditionType filter via concept set (ConceptSetSelection)

Usage

```
conditionTypeCS(conceptSet, isExclusion = FALSE)
```

Arguments

conceptSet	A ConceptSet built with cs().
isExclusion	logical; if TRUE the types in the concept set are excluded.

Value

An attribute for use in conditionOccurrence().

conditionTypeExclude	<i>ConditionTypeExclude query attribute</i>
----------------------	---

Description

When FALSE, condition type is not excluded (include all). When TRUE, excludes specified condition types (Capr does not support type-concept lists without DB).

Usage

```
conditionTypeExclude(exclude = FALSE)
```

Arguments

exclude	logical; FALSE = include all types (default), TRUE = exclude (unsupported in decompiler)
---------	--

Value

An attribute for use in conditionOccurrence()

continuousObservation *A function to construct the observationWindow*

Description

A function to construct the observationWindow

Usage

```
continuousObservation(priorDays = 0L, postDays = 0L)
```

Arguments

priorDays	minimum number of observation days prior to the cohort index. Default 0 days
postDays	minimum number of observation days post cohort index. Default 0 days

Value

An observation window that can be used in a Capr cohort definition

Criteria-class	<i>An S4 for a criteria</i>
----------------	-----------------------------

Description

a criteria is a temporal observation of a clinical event relative to the index event

Slots

occurrence	an occurrence object specifying how many events must occur to consider the event as part of the cohort definition
query	a query object that provides context to the clinical event of interest
aperture	an eventAperture object that shows the temporal span where the event is to be observed relative to the index event

cs

*Create a concept set***Description**

cs is used to create concept set expressions.

‘exclude’ is meant to be used inside ‘cs’ when creating a new concept set.

‘mapped’ is meant to be used inside ‘cs’ when creating a new concept set.

‘descendants’ is meant to be used inside ‘cs’ when creating a new concept set.

Usage

```
cs(..., name)
```

```
exclude(...)
```

```
mapped(...)
```

```
descendants(...)
```

Arguments

...	One or more numeric vectors that can be coerced to integers, or Calls to helper functions "exclude", "descendants", or "mapped".
name	A name for the concept set

Value

A Capr Concept Set Object

A list of Capr concepts

A list of Capr concepts

A list of Capr concepts

Functions

- `exclude()`: exclude concepts
- `mapped()`: Include mapped concepts
- `descendants()`: Include descendants

Examples

```
cs(1, 2, name = "concepts")
cs(1, c(10, 11, 2), name = "concepts")
cs(1, seq(2, 10, 2), name = "concepts")
cs(1, 2, 3, exclude(4, 5), name = "concepts")
cs(1, 2, 3, exclude(4, 5), mapped(6, 7), name = "concepts")
cs(1, 2, 3, exclude(4, 5), mapped(6, 7), descendants(8, 9), name = "concepts")
cs(descendants(1, 2, 3), exclude(descendants(8, 9)), name = "concepts")
```

dateAdjustment	<i>Shift the start and/or end date of a query event</i>
----------------	---

Description

Adds a DateAdjustment attribute to a domain query, shifting the event's effective start and/or end date relative to a reference anchor (START_DATE or END_DATE) and an offset in days. Used in Circe wherever DateAdjustment is supported.

Usage

```
dateAdjustment(
  startWith = "START_DATE",
  startOffset = 0L,
  endWith = "END_DATE",
  endOffset = 0L
)
```

Arguments

startWith	character; anchor for the adjusted start date, either "START_DATE" (default) or "END_DATE".
startOffset	integer offset in days added to the startWith anchor. Default 0.
endWith	character; anchor for the adjusted end date, either "END_DATE" (default) or "START_DATE".
endOffset	integer offset in days added to the endWith anchor. Default 0.

Details

For observationPeriod() and payerPlanPeriod(), dateAdjustment() also fulfils the role of Circe's UserDefinedPeriod: when a dateAdjustmentAttribute with name = "DateAdjustment" is present on an ObservationPeriod or PayerPlanPeriod query, Capr serializes it as UserDefinedPeriod { StartDate, EndDate } in the Circe JSON, which constrains the period to a caller-specified date window.

Value

A dateAdjustmentAttribute for use in any domain query constructor.

dateAdjustmentAttribute-class	<i>An S4 class for a date adjustment attribute</i>
-------------------------------	--

Description

An S4 class for a date adjustment attribute

Slots

name the name of the attribute
 startWith character string either START_DATE or END_DATE
 startOffset an integer value, default 0
 endWith character string either START_DATE or END_DATE
 endOffset an integer value, default 0

daysOfSupply	<i>Function to create days supply attribute</i>
--------------	---

Description

This function is used only for a drug query. days supply is a column in the drug exposure table of the cdm. This attribute allows a subquery to find drugs that satisfy certain values determined by the op logic.

Usage

```
daysOfSupply(op)
```

Arguments

op	an opAttribute object that is either numeric or integer that defines the logical operation used to determine eligible number of days of supply
----	--

Value

An attribute that can be used in a cohort definition

death	<i>Query the death domain</i>
-------	-------------------------------

Description

Query the death domain

Usage

```
death(conceptSet = NULL, ...)
```

Arguments

conceptSet	A ConceptSet for the death cause concept, or NULL (most common — death records are typically matched without a concept set).
...	Optional attributes: startDate(), deathType(), age(), etc.

Value

A Query object.

See Also

[censoringEvents](#), [exit](#)

Examples

```
# Any death (no concept set)
death(NULL)
```

deathType	<i>Add a death type attribute to determine the provenance of the record</i>
-----------	---

Description

Add a death type attribute to determine the provenance of the record

Usage

```
deathType(ids, connection = NULL, vocabularyDatabaseSchema = NULL)
```

Arguments

ids	integer concept ids for the attribute
connection	an optional connection to an OMOP CDM database, used only to look up concept names for display (e.g. in Atlas). When NULL (default) the attribute is built from the ids alone, which produces identical SQL.
vocabularyDatabaseSchema	the schema of the OMOP vocabulary tables; only used when 'connection' is supplied

Value

An attribute for use in `death()`.

deathTypeCS	<i>DeathType filter via concept set (ConceptSetSelection)</i>
-------------	---

Description

DeathType filter via concept set (ConceptSetSelection)

Usage

```
deathTypeCS(conceptSet, isExclusion = FALSE)
```

Arguments

conceptSet	A ConceptSet built with <code>cs()</code> .
isExclusion	logical; if TRUE the types in the concept set are excluded.

Value

An attribute for use in `death()`.

deathTypeExclude	<i>DeathTypeExclude query attribute</i>
------------------	---

Description

When FALSE, death type is not excluded (include all). When TRUE, excludes specified death types (Capr does not support type-concept lists without DB).

Usage

```
deathTypeExclude(exclude = FALSE)
```

Arguments

exclude	logical; FALSE = include all types (default), TRUE = exclude (unsupported in decompiler)
---------	--

Value

An attribute for use in death()

detectUnsupportedKeys	<i>Detect domain keys not in the supported set and emit skipOrStop for each (non-null/non-empty). Prevents silent semantic drift when new Circe fields appear.</i>
-----------------------	--

Description

Detect domain keys not in the supported set and emit skipOrStop for each (non-null/non-empty). Prevents silent semantic drift when new Circe fields appear.

Usage

```
detectUnsupportedKeys(
  domainKey,
  domainVal,
  supportedKeySet,
  emitter,
  jsonContextPath = ""
)
```

Arguments

domainKey	Character. The domain name (e.g. "ConditionOccurrence", "Measurement").
domainVal	Named list. The domain criterion object from the JSON.
supportedKeySet	Character vector of supported attribute keys for this domain.
emitter	Emitter object from makeEmitter() (handles skipOrStop).
jsonContextPath	Character. Path prefix for error messages (e.g. "PrimaryCriteria").

deviceExposure	<i>Query the device exposure domain</i>
----------------	---

Description

Query the device exposure domain

Usage

```
deviceExposure(conceptSet, ...)
```

Arguments

conceptSet	A ConceptSet built with cs() , or NULL for any device.
...	Optional attributes: firstOccurrence(), startDate(), endDate(), age(), deviceType(), uniqueDeviceId(), quantityValue(), etc.

Value

A Query object.

See Also

[entry](#), [cs](#)

deviceType	<i>Add a device type attribute to determine the provenance of the record</i>
------------	--

Description

Add a device type attribute to determine the provenance of the record

Usage

```
deviceType(ids, connection = NULL, vocabularyDatabaseSchema = NULL)
```

Arguments

ids	integer concept ids for the attribute
connection	an optional connection to an OMOP CDM database, used only to look up concept names for display (e.g. in Atlas). When NULL (default) the attribute is built from the ids alone, which produces identical SQL.
vocabularyDatabaseSchema	the schema of the OMOP vocabulary tables; only used when 'connection' is supplied

Value

An attribute for use in deviceExposure().

deviceTypeCS	<i>DeviceType filter via concept set (ConceptSetSelection)</i>
--------------	--

Description

DeviceType filter via concept set (ConceptSetSelection)

Usage

deviceTypeCS(conceptSet, isExclusion = FALSE)

Arguments

- conceptSet A ConceptSet built with cs().
- isExclusion logical; if TRUE the types in the concept set are excluded.

Value

An attribute for use in deviceExposure().

deviceTypeExclude	<i>DeviceTypeExclude query attribute</i>
-------------------	--

Description

DeviceTypeExclude query attribute

Usage

deviceTypeExclude(exclude = FALSE)

Arguments

- exclude logical; FALSE = include all types (default), TRUE = exclude

Value

An attribute for use in deviceExposure()

doseEra	<i>Query the dose era domain</i>
---------	----------------------------------

Description

Query the dose era domain

Usage

```
doseEra(conceptSet, ...)
```

Arguments

conceptSet	A drug ingredient concept set (optional)
...	optional attributes

Value

A Capr Query

doseUnit	<i>Dose unit attribute for drug exposure</i>
----------	--

Description

Dose unit attribute for drug exposure

Usage

```
doseUnit(ids, connection = NULL, vocabularyDatabaseSchema = NULL)
```

Arguments

ids	integer concept ids for the attribute
connection	an optional connection to an OMOP CDM database, used only to look up concept names for display (e.g. in Atlas). When NULL (default) the attribute is built from the ids alone, which produces identical SQL.
vocabularyDatabaseSchema	the schema of the OMOP vocabulary tables; only used when 'connection' is supplied

Value

An attribute for use in drugExposure().

doseUnitCS	<i>Dose unit filter via concept set — DrugExposure</i>
------------	--

Description

Dose unit filter via concept set — DrugExposure

Usage

```
doseUnitCS(conceptSet, isExclusion = FALSE)
```

Arguments

conceptSet	A ConceptSet built with <code>cs()</code> .
isExclusion	logical; if TRUE the types in the concept set are excluded.

Value

An attribute for use in `drugExposure()`.

doseValue	<i>Dose value attribute for DoseEra</i>
-----------	---

Description

Filter dose era criteria by dose value (e.g. `gt(92)` for dose > 92). Used only in a dose era query.

Usage

```
doseValue(op)
```

Arguments

op	an <code>opAttribute</code> object (numeric) that defines the logical operation and value
----	---

Value

An attribute for use in `doseEra()`

drugEra	<i>Query the drug era domain</i>
---------	----------------------------------

Description

Query the drug era domain

Usage

```
drugEra(conceptSet, ...)
```

Arguments

conceptSet	A ConceptSet for the drug ingredient.
...	Optional attributes: firstOccurrence(), startDate(), endDate(), eraLength(), occurrenceCount(), ageAtStart(), ageAtEnd(), etc.

Value

A Query object.

See Also

[drugExposure](#), [drugExit](#), [entry](#)

drugExit	<i>Function to create an exit based on exit based on the end of a continuous drug exposure</i>
----------	--

Description

Function to create an exit based on exit based on the end of a continuous drug exposure

Usage

```
drugExit(  
  conceptSet,  
  persistenceWindow = 0L,  
  surveillanceWindow = 0L,  
  daysSupplyOverride = NULL  
)
```

Arguments

conceptSet the concept set of the drug exposure used to identify the exit

persistenceWindow allow for a maximum of days between exposure records when inferring the era of persistence exposure

surveillanceWindow add days to the end of the era of persistence exposure as an additional period of surveillance prior to cohort exit

daysSupplyOverride force drug exposure days supply to a set number of days

Value

an S4 DrugExposureExit class which defines the cohort exit by end of drug exposure

drugExposure	<i>Query the drug exposure domain</i>
--------------	---------------------------------------

Description

Query the drug exposure domain

Usage

```
drugExposure(conceptSet, ...)
```

Arguments

conceptSet A ConceptSet built with [cs\(\)](#), or NULL for any drug.

... Optional attributes: [firstOccurrence\(\)](#), [startDate\(\)](#), [endDate\(\)](#), [age\(\)](#), [drugType\(\)](#), [drugRefills\(\)](#), [drugQuantity\(\)](#), [daysOfSupply\(\)](#), [lotNumber\(\)](#), [stopReason\(\)](#), [routeConcept\(\)](#), [drugSourceConcept\(\)](#), [nestedWithAll\(\)](#), etc.

Value

A Query object.

See Also

[drugEra](#), [entry](#), [cs](#)

Examples

```
metformin <- cs(descendants(1503297L), name = "Metformin")
drugExposure(metformin)
drugExposure(metformin, firstOccurrence(), daysOfSupply(gte(90L)))
```

DrugExposureExit-class

An S4 class for a cohort exit based on continuous exposure persistence.

Description

Specify a concept set that contains one or more drugs. A drug era will be derived from all drug exposure events for any of the drugs within the specified concept set, using the specified persistence window as a maximum allowable gap in days between successive exposure events and adding a specified surveillance window to the final exposure event. If no exposure event end date is provided, then an exposure event end date is inferred to be event start date + days supply in cases when days supply is available or event start date + 1 day otherwise. This event persistence assures that the cohort end date will be no greater than the drug era end date.

Slots

conceptSet the concept set of the drug exposure used to identify the exit

persistenceWindow allow for a maximum of days between exposure records when inferring the era of persistence exposure

surveillanceWindow add days to the end of the era of persistence exposure as an additional period of surveillance prior to cohort exit

daysSupplyOverride force drug exposure days supply to a set number of days

count an integer specifying the number of occurrences for a criteria

drugQuantity

Function to create quantity attribute

Description

This function is used only for a drug query. quantity is a column in the drug exposure table of the cdm. This attribute allows a subquery to find drugs that satisfy certain values determined by the op logic.

Usage

```
drugQuantity(op)
```

Arguments

op an opAttribute object that is either numeric or integer that defines the logical operation used to determine eligible quantity

Value

An attribute that can be used in a cohort definition

drugRefills	<i>Function to create refills attribute</i>
-------------	---

Description

This function is used only for a drug query. refills is a column in the drug exposure table of the cdm. This attribute allows a subquery to find drugs that satisfy certain values determined by the op logic.

Usage

```
drugRefills(op)
```

Arguments

op	an opAttribute object that is either numeric or integer that defines the logical operation used to determine eligible number of refills
----	---

Value

An attribute that can be used in a cohort definition

drugSourceConcept	<i>Add a drug source concept attribute</i>
-------------------	--

Description

Add a drug source concept attribute

Usage

```
drugSourceConcept(conceptSet)
```

Arguments

conceptSet	a ConceptSet object containing the source concepts
------------	--

Value

An attribute that can be used in a query function

drugType	<i>Add a drug type attribute to determine the provenance of the record</i>
----------	--

Description

Add a drug type attribute to determine the provenance of the record

Usage

```
drugType(ids, connection = NULL, vocabularyDatabaseSchema = NULL)
```

Arguments

ids	integer concept ids for the attribute
connection	an optional connection to an OMOP CDM database, used only to look up concept names for display (e.g. in Atlas). When NULL (default) the attribute is built from the ids alone, which produces identical SQL.
vocabularyDatabaseSchema	the schema of the OMOP vocabulary tables; only used when 'connection' is supplied

Value

An attribute for use in drugExposure().

drugTypeCS	<i>DrugType filter via concept set (ConceptSetSelection)</i>
------------	--

Description

DrugType filter via concept set (ConceptSetSelection)

Usage

```
drugTypeCS(conceptSet, isExclusion = FALSE)
```

Arguments

conceptSet	A ConceptSet built with cs().
isExclusion	logical; if TRUE the types in the concept set are excluded.

Value

An attribute for use in drugExposure().

drugTypeExclude	<i>DrugTypeExclude query attribute</i>
-----------------	--

Description

DrugTypeExclude query attribute

Usage

```
drugTypeExclude(exclude = FALSE)
```

Arguments

exclude	logical; FALSE = include all types (default), TRUE = exclude
---------	--

Value

An attribute for use in drugExposure()

duringInterval	<i>Function that creates an eventAperture an opening where an event can occur relative to the index event</i>
----------------	---

Description

Function that creates an eventAperture an opening where an event can occur relative to the index event

Usage

```
duringInterval(
  startWindow,
  endWindow = NULL,
  restrictVisit = FALSE,
  ignoreObservationPeriod = FALSE
)
```

Arguments

startWindow	the starting window where an event can occur
endWindow	the end window of where an event can occur. This parameter is optional
restrictVisit	a logical toggle specifying whether the event should occur on the same visit
ignoreObservationPeriod	a logical toggle specifying whether we can consider events outside the observation period

Value

An event aperture that can be used in a Capr cohort definition

endDate	<i>Function that creates a end date attribute</i>
---------	---

Description

Function that creates a end date attribute

Usage

```
endDate(op, type = "occurrence")
```

Arguments

op	an opAttribute object must be a date that defines the logical operation used to determine eligible end dates
type	specify the type of date to use either occurrence or era. default as occurrence

Value

An attribute that can be used in a cohort definition

Endpoint-class	<i>An S4 class for an Endpoint</i>
----------------	------------------------------------

Description

this determines the time in days relative to an index either before or after

Slots

days either a character string all or an integer for the number of days
 coeff a character string either before or after

entry	<i>Create a cohort entry definition</i>
-------	---

Description

Defines the index event(s) and required observation window for a cohort. Each query passed as ... is a candidate index event; a person enters the cohort at the first (or all, depending on primaryCriteriaLimit) qualifying event.

Usage

```
entry(
  ...,
  observationWindow = continuousObservation(0L, 0L),
  primaryCriteriaLimit = c("First", "All", "Last"),
  additionalCriteria = NULL,
  qualifiedLimit = NULL
)
```

Arguments

... One or more Query objects (e.g. `conditionOccurrence()`, `drugExposure()`).

observationWindow Minimum continuous observation before and after index. See [continuousObservation](#). Default `continuousObservation(0L, 0L)`.

primaryCriteriaLimit Whether to index on "First", "Last", or "All" qualifying events per person.

additionalCriteria An optional Group (withAll/withAny) further restricting which events qualify as the index. See [withAll](#).

qualifiedLimit Result limit after applying additionalCriteria. Required when additionalCriteria is non-NULL; defaults to primaryCriteriaLimit.

Value

A CohortEntry object.

See Also

[cohort](#), [attrition](#), [exit](#), [continuousObservation](#), [withAll](#)

Examples

```
t2dm <- cs(descendants(201826L), name = "T2DM")
entry(conditionOccurrence(t2dm), primaryCriteriaLimit = "First")
```

eq

Equal to operator

Description

Builds a comparison operator for use inside attribute functions such as [age](#), [valueAsNumber](#), [startDate](#), etc. Accepts integer, numeric, Date, or character inputs.

Usage

```
eq(x)

## S4 method for signature 'integer'
eq(x)

## S4 method for signature 'numeric'
eq(x)

## S4 method for signature 'Date'
eq(x)

## S4 method for signature 'character'
eq(x)
```

Arguments

x Comparison value (integer, numeric, Date, or character).

Value

An opAttribute for use inside attribute constructor functions.

See Also

[lt](#), [gt](#), [lte](#), [gte](#), [bt](#)

Examples

```
# Exactly 2 refills
drugExposure(cs(1L, name = "drug"), drugRefills(eq(2L)))
```

era	<i>Define cohort era collapse settings</i>
-----	--

Description

Controls how adjacent cohort episodes are collapsed (merged) and optionally constrains the study window. Episodes separated by fewer than eraDays days are merged.

Usage

```
era(eraDays = 0L, studyStartDate = NULL, studyEndDate = NULL)
```

Arguments

eraDays	Gap in days between consecutive episodes below which they are collapsed into a single era. Default 0L (no collapse).
studyStartDate	Optional date to truncate cohort membership before this date.
studyEndDate	Optional date to truncate cohort membership after this date.

Value

A CohortEra object.

See Also

[cohort](#)

Examples

```
era(eraDays = 30L)
era(eraDays = 99999L) # Occ cohort: collapse all episodes into one long era
```

eraLength	<i>Era length attribute for DrugEra</i>
-----------	---

Description

Filter drug era criteria by era length in days (e.g. `gt(14L)` for era longer than 14 days). Used only in a drug era query.

Usage

```
eraLength(op)
```

Arguments

`op` an opAttribute object (integer) that defines the logical operation and value

Value

An attribute for use in [drugEra\(\)](#)

ethnicityCS	<i>Ethnicity filter via concept set (ConceptSetSelection, for demographic criteria)</i>
-------------	---

Description

Ethnicity filter via concept set (ConceptSetSelection, for demographic criteria)

Usage

```
ethnicityCS(conceptSet, isExclusion = FALSE)
```

Arguments

`conceptSet` A ConceptSet built with `cs()`.
`isExclusion` logical; if TRUE the types in the concept set are excluded.

Value

An attribute for use in `withAll()/withAny()` attrition groups.

EventAperture-class	<i>An S4 class for Aperture</i>
---------------------	---------------------------------

Description

The aperture class provides context to when the criteria must be observed in a person timeline to pertain to the expression

Slots

startWindow a EventWindow class object identifying the start window
 endWindow a EventWindow class object identifying the end window (optional)
 restrictVisit a logic toggle where TRUE restricts to the same visit
 ignoreObservationPeriod a logic toggle where TRUE allows events outside the observation period

eventEnds	<i>Function creates an event window where the event ends</i>
-----------	--

Description

Function creates an event window where the event ends

Usage

```
eventEnds(a, b, index = c("startDate", "endDate"))
```

Arguments

a	the left side of the event window
b	the right side of the event window
index	specifying what part of the index we start looking for events either at the index start date or index enddate

Value

An event window that can be used in a Capr cohort definition

eventStarts	<i>Function creates an event window where the event starts</i>
-------------	--

Description

Function creates an event window where the event starts

Usage

```
eventStarts(a, b, index = c("startDate", "endDate"))
```

Arguments

a	the left side of the event window
b	the right side of the event window
index	specifying what part of the index we start looking for events either at the index start date or index enddate

Value

An event window that can be used in a Capr cohort definition

EventWindow-class	<i>An S4 class for a Window</i>
-------------------	---------------------------------

Description

A window class provides details on the end points of the timeline

Slots

event	a character string either start or end. Identifies the point of reference for the window
start	an endpoint object containing the days and coefficient for the start of the window
end	an endpoint object containing the days and coefficient for the end of the window
index	A character string either start or end. Identifies where the index is relative to the window

exactly	<i>Require exactly N occurrences of a query within a window</i>
---------	---

Description

Require exactly N occurrences of a query within a window

Usage

```
exactly(
  x,
  query,
  aperture = duringInterval(eventStarts(-Inf, Inf)),
  distinct = NA,
  countColumn = NA_character_
)
```

Arguments

x	Integer count of required occurrences.
query	A Query object (e.g. from <code>conditionOccurrence()</code>).
aperture	An EventAperture built with duringInterval . Defaults to all time.
distinct	If TRUE, count distinct values of countColumn.
countColumn	Column to use for distinct counting (e.g. "DOMAIN_CONCEPT").

Value

A Criteria object for use in [withAll](#), [withAny](#), etc.

See Also

[atLeast](#), [atMost](#), [duringInterval](#)

Examples

```
t2dm <- cs(descendants(201826L), name = "T2DM")
# No prior T2DM diagnosis at any time before index
exactly(0, conditionOccurrence(t2dm), duringInterval(eventStarts(-Inf, 0)))
```

exit	<i>Define cohort exit strategy</i>
------	------------------------------------

Description

Define cohort exit strategy

Usage

exit(endStrategy, censor = NULL)

Arguments

- endStrategy An end strategy object: [observationExit](#), [fixedExit](#), or [drugExit](#).
- censor Optional [censoringEvents](#) object of queries that end the cohort early if they occur after the index date.

Value

A CohortExit object.

See Also

[observationExit](#), [fixedExit](#), [drugExit](#), [censoringEvents](#), [cohort](#)

Examples

```
exit(endStrategy = observationExit())
exit(endStrategy = fixedExit(offsetDays = 30L))
```

firstOccurrence	<i>Add first occurrence attribute</i>
-----------------	---------------------------------------

Description

Add first occurrence attribute

Usage

firstOccurrence()

Value

An attribute that can be used in a query function

FixedDurationExit-class

An S4 class for a cohort exit based on fixed duration persistence.

Description

The event end date is derived from adding a number of days to the event's start or end date. If an offset is added to the event's start date, all cohort episodes will have the same fixed duration (subject to further censoring). If an offset is added to the event's end date, persons in the cohort may have varying cohort duration times due to the varying event durations (such as eras of persistent drug exposure or visit length of stay). This event persistence assures that the cohort end date will be no greater than the selected index event date, plus the days offset.

Slots

index specification of event date to offset

offsetDays an integer specifying the number of days to offset from the event date

fixedExit

Function to create an exit based on exit based on the end of a continuous drug exposure

Description

Function to create an exit based on exit based on the end of a continuous drug exposure

Usage

```
fixedExit(index = c("startDate", "endDate"), offsetDays)
```

Arguments

index specification of event date to offset. Can be either startDate or endDate

offsetDays an number specifying the days to offset from the event date. Will coerce to an integer

Value

a fixed Duration exit S4 object used to define the cohort exit as the end of a specified time

genderConcepts	<i>Add gender attribute with one or more concept IDs (e.g. both sexes)</i>
----------------	--

Description

Use for round-trip when JSON has Gender: [8507, 8532]; a single criterion yields one gender_concept_id in (...) in SQL instead of separate branches.

Usage

```
genderConcepts(...)
```

Arguments

... Integer concept IDs (e.g. 8507L, 8532L for male, female).

Value

A single conceptAttribute with name Gender for use in additional criteria.

genderCS	<i>Gender filter via concept set (ConceptSetSelection, for demographic criteria)</i>
----------	--

Description

Gender filter via concept set (ConceptSetSelection, for demographic criteria)

Usage

```
genderCS(conceptSet, isExclusion = FALSE)
```

Arguments

conceptSet A ConceptSet built with cs().
isExclusion logical; if TRUE the types in the concept set are excluded.

Value

An attribute for use in withAll()/withAny() attrition groups.

getConceptDescendants *Get all descendants of one or more standard concepts*

Description

Traverses concept_ancestor. SQL lives in inst/sql/getConceptDescendants.sql.

Usage

```
getConceptDescendants(
  conceptIds,
  connection,
  vocabularyDatabaseSchema,
  minLevels = 0L,
  maxLevels = Inf
)
```

Arguments

conceptIds	Integer vector of standard concept IDs.
connection	A DatabaseConnector connection to an OMOP CDM.
vocabularyDatabaseSchema	Schema containing the OMOP vocabulary tables. Required.
minLevels	Minimum levels of separation (default 0 includes seeds).
maxLevels	Maximum levels of separation (default Inf = all descendants).

Value

A tibble with concept details plus min/max_levels_of_separation.

See Also

searchConcepts, descendants, cs

Examples

```
## Not run:
connection <- DatabaseConnector::connect(Eunomia::getEunomiaConnectionDetails())
getConceptDescendants(313217L, connection, vocabularyDatabaseSchema = "main")

## End(Not run)
```

getConceptInfo	<i>Look up concept details by concept ID</i>
----------------	--

Description

SQL lives in inst/sql/getConceptInfo.sql.

Usage

```
getConceptInfo(conceptIds, connection, vocabularyDatabaseSchema)
```

Arguments

conceptIds	Integer vector of concept IDs.
connection	A DatabaseConnector connection to an OMOP CDM.
vocabularyDatabaseSchema	Schema containing the OMOP vocabulary tables. Required.

Value

A tibble with full concept table columns for the requested IDs.

See Also

getConceptSetDetails, searchConcepts

Examples

```
## Not run:
connection <- DatabaseConnector::connect(Eunomia::getEunomiaConnectionDetails())
getConceptInfo(c(313217L, 320128L), connection, vocabularyDatabaseSchema = "main")

## End(Not run)
```

getConceptSetCall	<i>Create the Capr code to build a concept set</i>
-------------------	--

Description

Create the Capr code to build a concept set

Usage

```
getConceptSetCall(x, name = x@Name)
```

Arguments

x	A concept set
name	the name of the concept set

Value

The Capr code required to build the concept set

getConceptSetDetails	<i>Fill in Concept Set details using a vocab</i>
----------------------	--

Description

Concept sets created in R using the ‘cs’ function do not contain details like "CONCEPT_NAME", "DOMAIN_ID", etc. If an OMOP CDM vocabulary is available then these details can be filled in by the ‘getConceptSetDetails’ function.

Usage

```
getConceptSetDetails(x, con, vocabularyDatabaseSchema = NULL)

## S4 method for signature 'ConceptSet'
getConceptSetDetails(x, con, vocabularyDatabaseSchema = NULL)

## S4 method for signature 'Cohort'
getConceptSetDetails(x, con, vocabularyDatabaseSchema = NULL)
```

Arguments

x	A concept set created by cs(), or a Cohort.
con	A connection to an OMOP CDM database
vocabularyDatabaseSchema	Schema name where your OMOP vocabulary format resides. Note that for SQL Server, this should include both the database and schema name, for example 'vocabulary.dbo'.

Details

Pass a ConceptSet to hydrate a single set, or a [Cohort](#) to hydrate every concept set in the cohort (entry events, attrition rules, censoring events, the drug-exit concept set, and concept-set attributes such as conditionSourceConcept()) in one call. Concept sets are hydrated once each, even when the same set is reused across the cohort.

Value

A modified version of the input (concept set or cohort) with concept details filled in.

Methods (by class)

- getConceptSetDetails(ConceptSet): Fill in the details for a single concept set
- getConceptSetDetails(Cohort): Hydrate every concept set in a cohort

Examples

```
## Not run:
# create a concept set
vocabularyDatabaseSchema = "cdm5"
anemia <- cs(descendants(439777,4013073,4013074), name = "anemia")

# fill in the details from an OMOP CDM
library(DatabaseConnector)
con <- connect(dbms = "postgresql", user = "postgres", password = "", server = "localhost/cdm")
anemia <- getConceptSetDetails(condition_anemia, con, vocabularyDatabaseSchema = "cdm5")

# hydrate every concept set in a cohort definition
cd <- cohort(conditionOccurrence(anemia))
cd <- getConceptSetDetails(cd, con, vocabularyDatabaseSchema = "cdm5")

## End(Not run)
```

Group-class	<i>An S4 class for a group</i>
-------------	--------------------------------

Description

a group is the combination of multiple criteria or sub groups

Slots

occurrence an occurrence object specifying how many events must occur to consider the event as part of the cohort definition

criteria a list of criteria that are grouped together

group a list of sub-groups to consider

gt	<i>Greater than operator</i>
----	------------------------------

Description

Builds a comparison operator for use inside attribute functions such as [age](#), [valueAsNumber](#), [startDate](#), etc. Accepts integer, numeric, or Date inputs; returns the matching opAttribute type.

Usage

```
gt(x)

## S4 method for signature 'integer'
gt(x)

## S4 method for signature 'numeric'
gt(x)

## S4 method for signature 'Date'
gt(x)
```

Arguments

x Comparison bound (integer, numeric, or Date).

Value

An opAttribute for use inside attribute constructor functions.

See Also

[lt](#), [gte](#), [lte](#), [eq](#), [bt](#)

Examples

```
# HbA1c > 6.5
measurement(cs(1L, name = "HbA1c"), valueAsNumber(gt(6.5)))
```

gte	<i>Greater than or equal to operator</i>
-----	--

Description

Builds a comparison operator for use inside attribute functions such as [age](#), [valueAsNumber](#), [startDate](#), etc. Accepts integer, numeric, or Date inputs; returns the matching opAttribute type.

Usage

```
gte(x)

## S4 method for signature 'integer'
gte(x)

## S4 method for signature 'numeric'
gte(x)

## S4 method for signature 'Date'
gte(x)
```

Arguments

x Comparison bound (integer, numeric, or Date).

Value

An opAttribute for use inside attribute constructor functions.

See Also

[lt](#), [gt](#), [lte](#), [eq](#), [bt](#)

Examples

```
# Age 18 or older
conditionOccurrence(cs(1L, name = "test"), age(gte(18L)))
```

insertCaprCode	<i>Insert decompiled Capr code into the current document (RStudio only)</i>
----------------	---

Description

Converts the given cohort JSON to Capr R code and inserts it into the active RStudio editor. The function looks upward from the current cursor position for the previous call to `insertCaprCode()` in the file and inserts the generated code on the line directly below that call. For interactive use in RStudio only; requires the **rstudioapi** package.

Usage

```
insertCaprCode(jsonPath, mode = c("strict", "skip"))
```

Arguments

<code>jsonPath</code>	Character. Path to the cohort JSON file.
<code>mode</code>	"strict" or "skip" (see jsonToCapr).

Value

Invisibly, the inserted Capr code (character string).

See Also

[jsonToCapr](#), [jsonToCaprFile](#)

jsonToCapr	<i>Decompile Atlas/Circe cohort JSON to Capr R code</i>
------------	---

Description

These functions convert a cohort definition exported from Atlas (Circe JSON) into equivalent Capr R code (concept sets and `cohort()` calls).

Usage

```
jsonToCapr(jsonPath, mode = c("strict", "skip"), returnSkipped = FALSE)
```

Arguments

<code>jsonPath</code>	Character. Path to the cohort JSON file.
<code>mode</code>	"strict" (default): stop on unsupported elements. "skip": omit unsupported parts and emit # SKIPPED: comments (unsafe).
<code>returnSkipped</code>	If TRUE, return a list with lines, skipped, and emptyGroupWarnings instead of just the character string (for regression/reporting).

Value

A single character string of R code with newlines between lines (so `cat(jsonToCapr(...))` prints nicely), or if `returnSkipped = TRUE`, a list with `lines`, `skipped`, `emptyGroupWarnings`. The generated code starts with `library(Capr)` so the file is self-contained and can be sourced directly.

Supported features

Concept sets (`cs()`), primary criteria and entry, additional criteria (`withAll/withAny`), inclusion rules (`attrition()`), correlated criteria (`nestedWithAll/nestedWithAny`), start/end windows (`duringInterval`), occurrence (`exactly/atLeast/atMost`), common attributes (`age`, `dates`, `firstOccurrence`, `dateAdjustment`, source concepts), Measurement/DrugExposure attributes, exit strategies (`fixedExit`, `drugExit`, `observationExit`), censoring criteria, and era collapse.

Unsupported (fail in strict mode or skip with `mode = "skip"`)

Domains: `Specimen`, `VisitDetail`; `VisitOccurrence.ProviderSpecialty`; any `*TypeExclude == TRUE` or `*Type` lists (Type lists require vocabulary lookup in `Capr`);

Unknown domain keys are reported via `detectUnsupportedKeys()` to avoid silent drift.

See Also

[cohort](#), [cs](#)

jsonToCaprFile

Write decompiled Capr R code to a file

Description

Write decompiled Capr R code to a file

Usage

```
jsonToCaprFile(jsonPath, outRPath, mode = c("strict", "skip"))
```

Arguments

<code>(jsonPath</code>	Character. Path to the cohort JSON file.
<code>outRPath</code>	Character. Path to the output R file (directory is created if needed).
<code>mode</code>	"strict" or "skip" (see jsonToCapr).

Value

Invisibly, `outRPath`.

keyValueAttribute-class	<i>Attribute that serializes to a single key-value pair in query JSON. Used for boolean query options like ConditionTypeExclude. Has a name slot (same as key) so compile/collectGuid work.</i>
-------------------------	---

Description

Attribute that serializes to a single key-value pair in query JSON. Used for boolean query options like ConditionTypeExclude. Has a name slot (same as key) so compile/collectGuid work.

logicAttribute-class	<i>An S4 class for a logical attribute</i>
----------------------	--

Description

with a logic attribute if it is specified than we assume it is true

Slots

name the name of the attribute

lotNumber	<i>Lot number attribute for DrugExposure</i>
-----------	--

Description

Filter drug exposure criteria by lot number (e.g. contains("LOT123") for lot numbers containing "LOT123"). Used only in a drug exposure query.

Usage

lotNumber(op)

Arguments

op	an opAttribute object (character) that defines the logical operation and value (e.g. contains("LOT") for lot numbers containing "LOT")
----	--

Value

An attribute for use in [drugExposure\(\)](#)

lt	<i>Less than operator</i>
----	---------------------------

Description

Builds a comparison operator for use inside attribute functions such as [age](#), [valueAsNumber](#), [startDate](#), etc. Accepts integer, numeric, or Date inputs; returns the matching opAttribute type.

Usage

```
lt(x)

## S4 method for signature 'integer'
lt(x)

## S4 method for signature 'numeric'
lt(x)

## S4 method for signature 'Date'
lt(x)
```

Arguments

x Comparison bound (integer, numeric, or Date).

Value

An opAttribute for use inside attribute constructor functions.

See Also

[gt](#), [gte](#), [lte](#), [eq](#), [bt](#), [age](#), [valueAsNumber](#)

Examples

```
# Patients under 18
conditionOccurrence(cs(1L, name = "test"), age(lt(18L)))
```

lte	<i>Less than or equal to operator</i>
-----	---------------------------------------

Description

Builds a comparison operator for use inside attribute functions such as [age](#), [valueAsNumber](#), [startDate](#), etc. Accepts integer, numeric, or Date inputs; returns the matching opAttribute type.

Usage

```
lte(x)

## S4 method for signature 'integer'
lte(x)

## S4 method for signature 'numeric'
lte(x)

## S4 method for signature 'Date'
lte(x)
```

Arguments

`x` Comparison bound (integer, numeric, or Date).

Value

An opAttribute for use inside attribute constructor functions.

See Also

[lt](#), [gt](#), [gte](#), [eq](#), [bt](#)

Examples

```
# Age 65 or younger
conditionOccurrence(cs(1L, name = "test"), age(lte(65L)))
```

makeCohortSet	<i>Make a cohort dataframe for cohort generator</i>
---------------	---

Description

Make a cohort dataframe for cohort generator

Usage

```
makeCohortSet(...)
```

Arguments

`...` multiple capr cohorts to bind into a dataframe

Value

a tibble containing cohortId, name, sql and json to pipe into CohortGenerator.

male	<i>Add male attribute to a query</i>
------	--------------------------------------

Description

Add male attribute to a query
Add female attribute to a query
Add race filter to a demographic criteria group
Add ethnicity filter to a demographic criteria group

Usage

```
male()  
  
female()  
  
raceConcepts(...)  
  
ethnicityConcepts(...)
```

Arguments

... Integer OMOP ethnicity concept IDs (e.g. 38003563L Hispanic or Latino, 38003564L Not Hispanic or Latino).

Value

An attribute that can be used in a query function
An attribute that can be used in a query function
A conceptAttribute with name Race for use in withAll()/withAny() attrition groups.
A conceptAttribute with name Ethnicity for use in withAll()/withAny() attrition groups.

Functions

- male(): male demographic attribute
- female(): female demographic attribute
- raceConcepts(): race demographic attribute
- ethnicityConcepts(): ethnicity demographic attribute

Examples

```
# Create a cohort of males with Type 1 diabetes  
t1dm <- cs(descendants(201254, 435216, 40484648), name = "type 1 diabetes")  
t1dm_males <- cohort(conditionOccurrence(t1dm, male()))  
# Create a cohort of males with Type 1 diabetes  
t1dm <- cs(descendants(201254, 435216, 40484648), name = "type 1 diabetes")  
t1dm_females <- cohort(conditionOccurrence(t1dm, female()))
```

mapSourceToStandard	<i>Map source (non-standard) codes to their standard OMOP concepts</i>
---------------------	--

Description

Uses the concept_relationship 'Maps to' relationship. SQL lives in inst/sql/mapSourceToStandard.sql.

Usage

```
mapSourceToStandard(
  sourceCodes,
  connection,
  vocabularyDatabaseSchema,
  vocabularyId = NULL
)
```

Arguments

sourceCodes	Character vector of source codes (e.g. ICD-10 codes).
connection	A DatabaseConnector connection to an OMOP CDM.
vocabularyDatabaseSchema	Schema containing the OMOP vocabulary tables. Required.
vocabularyId	Optional vocabulary IDs to restrict source lookup (e.g. "ICD10CM").

Value

A tibble with source and mapped standard concept columns.

See Also

searchConcepts, cs, mapped

Examples

```
## Not run:
connection <- DatabaseConnector::connect(Eunomia::getEunomiaConnectionDetails())
mapSourceToStandard(c("I48", "I48.0"), connection,
  vocabularyDatabaseSchema = "main", vocabularyId = "ICD10CM")

## End(Not run)
```

measurement	<i>Query the measurement domain</i>
-------------	-------------------------------------

Description

Query the measurement domain

Usage

```
measurement(conceptSet, ...)
```

Arguments

conceptSet	A ConceptSet built with cs() , or NULL for any measurement.
...	Optional attributes: firstOccurrence() , startDate() , valueAsNumber() , valueAsConcept() , measurementUnit() , measurementType() , rangeLow() , rangeHigh() , measurementSourceConcept() , age() , etc.

Value

A Query object.

See Also

[valueAsNumber](#), [measurementUnit](#), [entry](#)

Examples

```
hba1c <- cs(descendants(4184637L), name = "HbA1c")
measurement(hba1c, valueAsNumber(gt(6.5)), measurementUnit(8554L))
```

measurementOperator	<i>Measurement operator attribute (e.g. equal to, less than)</i>
---------------------	--

Description

Measurement operator attribute (e.g. equal to, less than)

Usage

```
measurementOperator(ids, connection = NULL, vocabularyDatabaseSchema = NULL)
```

Arguments

ids	integer concept ids for the attribute
connection	an optional connection to an OMOP CDM database, used only to look up concept names for display (e.g. in Atlas). When NULL (default) the attribute is built from the ids alone, which produces identical SQL.
vocabularyDatabaseSchema	the schema of the OMOP vocabulary tables; only used when 'connection' is supplied

Value

An attribute for use in measurement().

measurementOperatorCS	<i>Measurement operator filter via concept set</i>
-----------------------	--

Description

Measurement operator filter via concept set

Usage

measurementOperatorCS(conceptSet, isExclusion = FALSE)

Arguments

- conceptSet A ConceptSet built with cs().
- isExclusion logical; if TRUE the types in the concept set are excluded.

Value

An attribute for use in measurement().

measurementSourceConcept	<i>Add a measurement source concept attribute</i>
--------------------------	---

Description

Add a measurement source concept attribute

Usage

measurementSourceConcept(conceptSet)

Arguments

- conceptSet a ConceptSet object containing the source concepts

Value

An attribute that can be used in a measurement query

measurementType	<i>Add a measurement type attribute to determine the provenance of the record</i>
-----------------	---

Description

Add a measurement type attribute to determine the provenance of the record

Usage

```
measurementType(ids, connection = NULL, vocabularyDatabaseSchema = NULL)
```

Arguments

ids	integer concept ids for the attribute
connection	an optional connection to an OMOP CDM database, used only to look up concept names for display (e.g. in Atlas). When NULL (default) the attribute is built from the ids alone, which produces identical SQL.
vocabularyDatabaseSchema	the schema of the OMOP vocabulary tables; only used when 'connection' is supplied

Value

An attribute for use in `measurement()`.

measurementTypeCS	<i>MeasurementType filter via concept set (ConceptSetSelection)</i>
-------------------	---

Description

MeasurementType filter via concept set (ConceptSetSelection)

Usage

```
measurementTypeCS(conceptSet, isExclusion = FALSE)
```

Arguments

conceptSet	A ConceptSet built with <code>cs()</code> .
isExclusion	logical; if TRUE the types in the concept set are excluded.

Value

An attribute for use in `measurement()`.

measurementTypeExclude

MeasurementTypeExclude query attribute

Description

When FALSE, measurement type is not excluded (include all). When TRUE, excludes specified measurement types (Capr does not support type-concept lists without DB).

Usage

```
measurementTypeExclude(exclude = FALSE)
```

Arguments

exclude	logical; FALSE = include all types (default), TRUE = exclude (unsupported in decompiler)
---------	--

Value

An attribute for use in measurement()

measurementUnit

Unit attribute for a query

Description

Filters by unit concept id (serializes to Circe Unit field). Applies to measurement(), observation(), doseEra(), and specimen().

Usage

```
measurementUnit(ids, connection = NULL, vocabularyDatabaseSchema = NULL)
```

Arguments

ids	integer concept ids for the attribute
connection	an optional connection to an OMOP CDM database, used only to look up concept names for display (e.g. in Atlas). When NULL (default) the attribute is built from the ids alone, which produces identical SQL.
vocabularyDatabaseSchema	the schema of the OMOP vocabulary tables; only used when 'connection' is supplied

Details

Named measurementUnit (not unit) to avoid masking dplyr::unit.

Value

An attribute for use in measurement(), observation(), doseEra(), or specimen().

nbt	<i>Not between operator</i>
-----	-----------------------------

Description

Builds an exclusion range comparison for use inside attribute functions. Accepts integer, numeric, or Date inputs.

Usage

```
nbt(x, y)

## S4 method for signature 'integer'
nbt(x, y)

## S4 method for signature 'numeric'
nbt(x, y)

## S4 method for signature 'Date'
nbt(x, y)
```

Arguments

x	Lower bound (integer, numeric, or Date).
y	Upper bound; must match type of x.

Value

An opAttribute for use inside attribute constructor functions.

See Also

[bt](#), [lt](#), [gt](#)

Examples

```
# Age outside 18-65
conditionOccurrence(cs(1L, name = "test"), age(nbt(18L, 65L)))
```

nestedAttribute-class	<i>An S4 class for a nested attribute</i>
-----------------------	---

Description

An S4 class for a nested attribute

Slots

name	the name of the attribute
conceptSet	a list representing the concepts for the attribute

nestedWithAll	<i>Construct a nested group where all criteria and groups must be satisfied</i>
---------------	---

Description

Construct a nested group where all criteria and groups must be satisfied

Usage

```
nestedWithAll(...)
```

Arguments

...	a set of criteria or groups
-----	-----------------------------

nestedWithAny	<i>Function to construct a nested group where any criteria and groups may be satisfied</i>
---------------	--

Description

Function to construct a nested group where any criteria and groups may be satisfied

Usage

```
nestedWithAny(...)
```

Arguments

...	a set of criteria or groups
-----	-----------------------------

nestedWithAtLeast	<i>Function to construct a nested group where at least some of the criteria or groups must be satisfied</i>
-------------------	---

Description

Function to construct a nested group where at least some of the criteria or groups must be satisfied

Usage

```
nestedWithAtLeast(x, ...)
```

Arguments

x	an integer specifying the number of criteria or groups that must be satisfied
...	a set of criteria or groups

nestedWithAtMost	<i>Function to construct a nested group where at most some of the criteria or groups must be satisfied</i>
------------------	--

Description

Function to construct a nested group where at most some of the criteria or groups must be satisfied

Usage

```
nestedWithAtMost(x, ...)
```

Arguments

x	an integer specifying the number of criteria or groups that must be satisfied
...	a set of criteria or groups

observation	<i>Query the observation domain</i>
-------------	-------------------------------------

Description

Query the observation domain

Usage

```
observation(conceptSet, ...)
```

Arguments

conceptSet	A ConceptSet built with cs() , or NULL for any observation.
...	Optional attributes: firstOccurrence() , startDate() , valueAsNumber() , valueAsString() , valueAsConcept() , observationType() , observationQualifier() , observationSourceConcept() , age() , etc.

Value

A Query object.

See Also

[entry](#), [cs](#)

observationExit	<i>Function to create an exit based on continuous observation</i>
-----------------	---

Description

Function to create an exit based on continuous observation

Usage

observationExit()

Value

an S4 ObservationExit class which defines the cohort exit as the end of continuous observation

ObservationExit-class	<i>An S4 class for a cohort exit based on end of continuous observation.</i>
-----------------------	--

Description

The event persists until the end of continuous observation of the persons

Slots

index specification of event date to offset

offsetDays an integer specifying the number of days to offset from the event date

observationPeriod	<i>Query the observation period domain</i>
-------------------	--

Description

Query the observation period domain

Usage

observationPeriod(...)

Arguments

... optional attributes

Value

A Capr Query of domain observation period

observationPeriodType *Add a observation period type attribute to determine the provenance of the record*

Description

Add a observation period type attribute to determine the provenance of the record

Usage

```
observationPeriodType(ids, connection = NULL, vocabularyDatabaseSchema = NULL)
```

Arguments

ids	integer concept ids for the attribute
connection	an optional connection to an OMOP CDM database, used only to look up concept names for display (e.g. in Atlas). When NULL (default) the attribute is built from the ids alone, which produces identical SQL.
vocabularyDatabaseSchema	the schema of the OMOP vocabulary tables; only used when 'connection' is supplied

Value

An attribute that can be used in a query function

observationQualifier *Observation qualifier attribute*

Description

Observation qualifier attribute

Usage

```
observationQualifier(ids, connection = NULL, vocabularyDatabaseSchema = NULL)
```

Arguments

ids	integer concept ids for the attribute
connection	an optional connection to an OMOP CDM database, used only to look up concept names for display (e.g. in Atlas). When NULL (default) the attribute is built from the ids alone, which produces identical SQL.
vocabularyDatabaseSchema	the schema of the OMOP vocabulary tables; only used when 'connection' is supplied

Value

An attribute for use in observation().

`observationQualifierCS`*Observation qualifier filter via concept set*

Description

Observation qualifier filter via concept set

Usage

```
observationQualifierCS(conceptSet, isExclusion = FALSE)
```

Arguments

`conceptSet` A `ConceptSet` built with `cs()`.

`isExclusion` logical; if `TRUE` the types in the concept set are excluded.

Value

An attribute for use in `observation()`.

`observationSourceConcept`*Add a observation source concept attribute*

Description

Add a observation source concept attribute

Usage

```
observationSourceConcept(conceptSet)
```

Arguments

`conceptSet` a `ConceptSet` object containing the source concepts

Value

An attribute that can be used in a query function

observationType	<i>Add an observation type attribute to determine the provenance of the record</i>
-----------------	--

Description

Add an observation type attribute to determine the provenance of the record

Usage

```
observationType(ids, connection = NULL, vocabularyDatabaseSchema = NULL)
```

Arguments

ids	integer concept ids for the attribute
connection	an optional connection to an OMOP CDM database, used only to look up concept names for display (e.g. in Atlas). When NULL (default) the attribute is built from the ids alone, which produces identical SQL.
vocabularyDatabaseSchema	the schema of the OMOP vocabulary tables; only used when 'connection' is supplied

Value

An attribute for use in `observation()`.

observationTypeCS	<i>ObservationType filter via concept set (ConceptSetSelection)</i>
-------------------	---

Description

ObservationType filter via concept set (ConceptSetSelection)

Usage

```
observationTypeCS(conceptSet, isExclusion = FALSE)
```

Arguments

conceptSet	A ConceptSet built with <code>cs()</code> .
isExclusion	logical; if TRUE the types in the concept set are excluded.

Value

An attribute for use in `observation()`.

observationTypeExclude

ObservationTypeExclude query attribute

Description

ObservationTypeExclude query attribute

Usage

observationTypeExclude(exclude = FALSE)

Arguments

exclude logical; FALSE = include all types (default), TRUE = exclude

Value

An attribute for use in observation()

ObservationWindow-class

An S4 class for an ObservationWindow

Description

this determines the minimal observation time before and after index for all persons in the cohort

Slots

priorDays minimum number of days prior to the cohort index

postDays minimum number of days post cohort index

Occurrence-class

An S4 class for an occurrence.

Description

This determines how many events need to occur to count the criteria in the cohort definition (relative to the index event)

Slots

type a character string determine the logic for counting occurrences. Can be all, any, exactly, atLeast, or atMost

count an integer specifying the number of occurrences for a criteria

isDistinct optional logical; if TRUE, count distinct (e.g. by CountColumn)

countColumn optional character; when counting, use this column for distinct (e.g. "DOMAIN_CONCEPT")

occurrenceCount	<i>Occurrence count attribute for ConditionEra</i>
-----------------	--

Description

Filter condition era criteria by the number of condition eras (e.g. occurrence count = 0 for no matching eras). Used only in a condition era query.

Usage

```
occurrenceCount(op)
```

Arguments

op	an opAttribute object (integer) that defines the logical operation and value (e.g. eq(0L) for "count equals 0")
----	---

Value

An attribute for use in `conditionEra()`

opAttributeCharacter-class	<i>An S4 class for a op attribute that is a character</i>
----------------------------	---

Description

An S4 class for a op attribute that is a character

Slots

name	the name of the attribute
op	the operator one of: startsWith, contains, endsWith, !startsWith, !contains, !endsWith
value	a value or pattern for comparison
extent	unused for character attributes

opAttributeDate-class	<i>An S4 class for a op attribute that is a date</i>
-----------------------	--

Description

An S4 class for a op attribute that is a date

Slots

name	the name of the attribute
op	the operator one of: gt,lt,gte,lte,eq,bt,!bt
value	a value serving as the single limit or lower limit in a bt.
extent	a value serving as the upper limit in a bt, otherwise this is empty

opAttributeInteger-class

An S4 class for a op attribute that is an integer

Description

An S4 class for a op attribute that is an integer

Slots

name the name of the attribute

op the operator one of: gt,lt,gte,lte,eq,bt,!bt

value a value serving as the single limit or lower limit in a bt.

extent a value serving as the upper limit in a bt, otherwise this is empty

opAttributeNumeric-class

An S4 class for a op attribute that is a numeric

Description

An S4 class for a op attribute that is a numeric

Slots

name the name of the attribute

op the operator one of: gt,lt,gte,lte,eq,bt,!bt

value a value serving as the single limit or lower limit in a bt

extent a value serving as the upper limit in a bt, otherwise this is empty

opAttributeSuper-class

An S4 super class for other opAttribute objects to inherit.

Description

An S4 super class for other opAttribute objects to inherit.

payerConcept	<i>Payer concept attribute for PayerPlanPeriod</i>
--------------	--

Description

Payer concept attribute for PayerPlanPeriod

Usage

```
payerConcept(conceptSet)
```

Arguments

conceptSet	A ConceptSet for the payer concept filter.
------------	--

Value

An attribute for use in payerPlanPeriod().

payerPlanPeriod	<i>Query the payer plan period domain (CDM >= 5.3)</i>
-----------------	---

Description

Query the payer plan period domain (CDM >= 5.3)

Usage

```
payerPlanPeriod(...)
```

Arguments

...	optional attributes (e.g. firstOccurrence(), startDate(), periodLength(), ageAtStart(), ageAtEnd(), genderCS())
-----	---

Value

A Capr Query of domain PayerPlanPeriod

payerSourceConcept	<i>Payer source concept attribute for PayerPlanPeriod</i>
--------------------	---

Description

Payer source concept attribute for PayerPlanPeriod

Usage

payerSourceConcept(conceptSet)

Arguments

conceptSet	A ConceptSet for the payer source concept filter.
------------	---

Value

An attribute for use in payerPlanPeriod().

periodLength	<i>Period length attribute — ObservationPeriod, PayerPlanPeriod</i>
--------------	---

Description

Period length attribute — ObservationPeriod, PayerPlanPeriod

Usage

periodLength(op)

Arguments

op	an opAttribute object (integer or numeric) defining the length comparison in days.
----	--

Value

An attribute for use in observationPeriod() or payerPlanPeriod().

periodTypeCS	<i>ObservationPeriod period type filter via concept set (ConceptSetSelection)</i>
--------------	---

Description

ObservationPeriod period type filter via concept set (ConceptSetSelection)

Usage

```
periodTypeCS(conceptSet, isExclusion = FALSE)
```

Arguments

conceptSet	A ConceptSet built with cs().
isExclusion	logical; if TRUE the types in the concept set are excluded.

Value

An attribute for use in observationPeriod().

placeOfService	<i>Place of service attribute for visit occurrence</i>
----------------	--

Description

Place of service attribute for visit occurrence

Usage

```
placeOfService(ids, connection = NULL, vocabularyDatabaseSchema = NULL)
```

Arguments

ids	integer concept ids for the attribute
connection	an optional connection to an OMOP CDM database, used only to look up concept names for display (e.g. in Atlas). When NULL (default) the attribute is built from the ids alone, which produces identical SQL.
vocabularyDatabaseSchema	the schema of the OMOP vocabulary tables; only used when 'connection' is supplied

Value

An attribute for use in visit().

placeOfServiceCS	<i>Place of service filter via concept set — VisitOccurrence, VisitDetail</i>
------------------	---

Description

Place of service filter via concept set — VisitOccurrence, VisitDetail

Usage

```
placeOfServiceCS(conceptSet, isExclusion = FALSE)
```

Arguments

conceptSet	A ConceptSet built with cs().
isExclusion	logical; if TRUE the types in the concept set are excluded.

Value

An attribute for use in visit() or visitDetail().

planConcept	<i>Plan concept attribute for PayerPlanPeriod</i>
-------------	---

Description

Plan concept attribute for PayerPlanPeriod

Usage

```
planConcept(conceptSet)
```

Arguments

conceptSet	A ConceptSet for the plan concept filter.
------------	---

Value

An attribute for use in payerPlanPeriod().

planSourceConcept	<i>Plan source concept attribute for PayerPlanPeriod</i>
-------------------	--

Description

Plan source concept attribute for PayerPlanPeriod

Usage

planSourceConcept(conceptSet)

Arguments

conceptSet A ConceptSet for the plan source concept filter.

Value

An attribute for use in payerPlanPeriod().

procedure	<i>Query the procedure occurrence domain</i>
-----------	--

Description

Query the procedure occurrence domain

Usage

procedure(conceptSet, ...)

Arguments

conceptSet A ConceptSet built with [cs\(\)](#), or NULL for any procedure.
... Optional attributes: firstOccurrence(), startDate(), age(), procedureType(),
procedureModifier(), quantityValue(), procedureSourceConcept(), etc.

Value

A Query object.

See Also

[entry](#), [cs](#)

procedureModifier	<i>Procedure modifier attribute</i>
-------------------	-------------------------------------

Description

Procedure modifier attribute

Usage

```
procedureModifier(ids, connection = NULL, vocabularyDatabaseSchema = NULL)
```

Arguments

ids	integer concept ids for the attribute
connection	an optional connection to an OMOP CDM database, used only to look up concept names for display (e.g. in Atlas). When NULL (default) the attribute is built from the ids alone, which produces identical SQL.
vocabularyDatabaseSchema	the schema of the OMOP vocabulary tables; only used when 'connection' is supplied

Value

An attribute for use in procedure().

procedureModifierCS	<i>Procedure modifier filter via concept set</i>
---------------------	--

Description

Procedure modifier filter via concept set

Usage

```
procedureModifierCS(conceptSet, isExclusion = FALSE)
```

Arguments

conceptSet	A ConceptSet built with cs().
isExclusion	logical; if TRUE the types in the concept set are excluded.

Value

An attribute for use in procedure().

```
procedureSourceConcept
```

Add a procedure source concept attribute

Description

Add a procedure source concept attribute

Usage

```
procedureSourceConcept(conceptSet)
```

Arguments

conceptSet a ConceptSet object containing the source concepts

Value

An attribute that can be used in a query function

```
procedureType
```

Add a procedure type attribute to determine the provenance of the record

Description

Add a procedure type attribute to determine the provenance of the record

Usage

```
procedureType(ids, connection = NULL, vocabularyDatabaseSchema = NULL)
```

Arguments

ids integer concept ids for the attribute

connection an optional connection to an OMOP CDM database, used only to look up concept names for display (e.g. in Atlas). When NULL (default) the attribute is built from the ids alone, which produces identical SQL.

vocabularyDatabaseSchema
the schema of the OMOP vocabulary tables; only used when 'connection' is supplied

Value

An attribute for use in procedure().

procedureTypeCS	<i>ProcedureType filter via concept set (ConceptSetSelection)</i>
-----------------	---

Description

ProcedureType filter via concept set (ConceptSetSelection)

Usage

procedureTypeCS(conceptSet, isExclusion = FALSE)

Arguments

- conceptSet A ConceptSet built with cs().
- isExclusion logical; if TRUE the types in the concept set are excluded.

Value

An attribute for use in procedure().

procedureTypeExclude	<i>ProcedureTypeExclude query attribute</i>
----------------------	---

Description

ProcedureTypeExclude query attribute

Usage

procedureTypeExclude(exclude = FALSE)

Arguments

- exclude logical; FALSE = include all types (default), TRUE = exclude

Value

An attribute for use in procedure()

 providerSpecialtyConcepts

Add provider specialty filter to a visit (round-trip from Atlas JSON)

Description

When JSON has VisitOccurrence.ProviderSpecialty: [{ CONCEPT_ID: ... }], use this so round-trip preserves the PR.specialty_concept_id in (...) filter.

Usage

```
providerSpecialtyConcepts(...)
```

Arguments

... Integer concept IDs for provider specialty (e.g. 38004463L).

Value

A conceptAttribute with name ProviderSpecialty for use in visit().

 providerSpecialtyCS

Provider specialty filter via concept set — multiple domains

Description

Provider specialty filter via concept set — multiple domains

Usage

```
providerSpecialtyCS(conceptSet, isExclusion = FALSE)
```

Arguments

conceptSet A ConceptSet built with cs().

isExclusion logical; if TRUE the types in the concept set are excluded.

Value

An attribute for use in query functions that support provider specialty.

quantityValue	<i>Quantity attribute — ProcedureOccurrence, DeviceExposure, Specimen</i>
---------------	---

Description

Quantity attribute — ProcedureOccurrence, DeviceExposure, Specimen

Usage

quantityValue(op)

Arguments

op an opAttribute object (numeric or integer) defining the quantity comparison.

Value

An attribute for use in procedure(), deviceExposure(), or specimen().

Query-class	<i>An S4 class for a Circe Query</i>
-------------	--------------------------------------

Description

A query is a medical concept that can be extracted from a database through a 'where' clause in a 'SQL' statement. This includes concepts.

Slots

domain The domain to search (e.g. "Condition", "Drug", "Measurement", etc)
conceptSet The Concept set describing the observation to serach for
attributes a list of attributes that modify the query (e.g. 'male()', 'female()', 'age(gte(65))')

raceCS	<i>Race filter via concept set (ConceptSetSelection, for demographic criteria)</i>
--------	--

Description

Race filter via concept set (ConceptSetSelection, for demographic criteria)

Usage

raceCS(conceptSet, isExclusion = FALSE)

Arguments

conceptSet	A ConceptSet built with cs().
isExclusion	logical; if TRUE the types in the concept set are excluded.

Value

An attribute for use in withAll()/withAny() attrition groups.

rangeHigh	<i>Function to create rangeHigh attribute</i>
-----------	---

Description

This function is used only for measurement query. range_high is a column in the measurement table of the cdm. This attribute allows a subquery to find measurements that satisfy certain values determined by the op logic.

Usage

```
rangeHigh(op)
```

Arguments

op	an opAttribute object that is either numeric or integer that defines the logical operation used to determine eligible range high
----	--

Value

An attribute that can be used in a cohort definition

rangeHighRatio	<i>Range high ratio attribute for Measurement</i>
----------------	---

Description

Filter measurement criteria by the ratio value_as_number / range_high (e.g. gt(2) for ratio > 2). Used only in a measurement query.

Usage

```
rangeHighRatio(op)
```

Arguments

op	an opAttribute object (numeric) that defines the logical operation and value
----	--

Value

An attribute for use in `measurement()`

rangeLow	<i>Function to create rangeLow attribute</i>
----------	--

Description

This function is used only for measurement query. range_low is a column in the measurement table of the cdm. This attribute allows a subquery to find measurements that satisfy certain values determined by the op logic.

Usage

```
rangeLow(op)
```

Arguments

op	an opAttribute object that is either numeric or integer that defines the logical operation used to determine eligible range low
----	---

Value

An attribute that can be used in a cohort definition

rankedSearchConcepts	<i>Ranked vocabulary search with dialect-specific similarity scoring</i>
----------------------	--

Description

A two-phase search that first narrows candidates with ILIKE, then ranks them using the best similarity function available for the connected DBMS:

- **PostgreSQL:** similarity() from pg_trgm (requires the extension).
- **Snowflake:** JAROWINKLER_SIMILARITY().
- **Spark / Databricks:** normalized levenshtein().
- **All other dialects:** positional boost scoring (exact > prefix > contains).

Synonym matching and mapping-count enrichment are scoped to the candidate set, avoiding a full-table synonym scan.

Usage

```
rankedSearchConcepts(
  keyword,
  connection,
  vocabularyDatabaseSchema,
  domain = NULL,
  standardOnly = TRUE,
  limit = 50L,
  offset = 0L
)
```

Arguments

keyword	Character string to search for.
connection	A DatabaseConnector connection to an OMOP CDM.
vocabularyDatabaseSchema	Schema containing the OMOP vocabulary tables. Required.
domain	Optional character vector restricting by domain_id (e.g. "Condition").
standardOnly	If TRUE (default), return only standard concepts.
limit	Maximum rows to return. Default 50L.
offset	Pagination offset. Default 0L.

Details

For PostgreSQL the GIN trigram index dramatically improves performance but is not required:
 CREATE INDEX ON concept USING GIN (concept_name gin_trgm_ops);.

Value

A tibble with columns: concept_id, concept_name, concept_code, vocabulary_id, domain_id, concept_class_id, standard_concept, relevance (0–1.5 float), mapping_count.

See Also

[searchConcepts](#), [cs](#)

Examples

```
## Not run:
connection <- DatabaseConnector::connect(Eunomia::getEunomiaConnectionDetails())
rankedSearchConcepts("atrial fibrillation", connection, vocabularyDatabaseSchema = "main")
rankedSearchConcepts("metformin", connection, vocabularyDatabaseSchema = "main",
  domain = "Drug", limit = 20L)

## End(Not run)
```

readConceptSet	<i>Read a concept set json or csv into R</i>
----------------	--

Description

Concept sets can be serialized to json or csv file formats. ‘readConceptSet’ reads the files into R as Capr concepts sets.

Usage

```
readConceptSet(path, name)
```

Arguments

path	Name of concept set file to read in csv or json format. (e.g. "concepts.json")
name	the name of the concept set

Examples

```
library(Capr)
path <- tempfile("concepts", fileext = ".json")
concepts <- cs(1, 2, descendants(4, 5), exclude(descendants(6, 7)), name = "test")
writeConceptSet(concepts, path = path)

concepts <- readConceptSet(path)
```

routeConcept	<i>Route concept attribute for drug exposure</i>
--------------	--

Description

Route concept attribute for drug exposure

Usage

```
routeConcept(ids, connection = NULL, vocabularyDatabaseSchema = NULL)
```

Arguments

- ids integer concept ids for the attribute
- connection an optional connection to an OMOP CDM database, used only to look up concept names for display (e.g. in Atlas). When NULL (default) the attribute is built from the ids alone, which produces identical SQL.
- vocabularyDatabaseSchema the schema of the OMOP vocabulary tables; only used when 'connection' is supplied

Value

An attribute for use in drugExposure().

routeConceptCS	<i>Route concept filter via concept set — DrugExposure</i>
----------------	--

Description

Route concept filter via concept set — DrugExposure

Usage

```
routeConceptCS(conceptSet, isExclusion = FALSE)
```

Arguments

- conceptSet A ConceptSet built with cs().
- isExclusion logical; if TRUE the types in the concept set are excluded.

Value

An attribute for use in drugExposure().

searchConcepts	<i>Search the OMOP vocabulary by keyword</i>
----------------	--

Description

Returns concepts whose concept_name matches a keyword. Search is case-insensitive. SQL lives in inst/sql/searchConcepts.sql.

Usage

```
searchConcepts(
  keyword,
  connection,
  vocabularyDatabaseSchema,
  domain = NULL,
  standardOnly = TRUE,
  limit = 100L
)
```

Arguments

keyword	Character string to search for (partial match).
connection	A DatabaseConnector connection to an OMOP CDM.
vocabularyDatabaseSchema	Schema containing the OMOP vocabulary tables. Required
domain	Optional character vector restricting by domain_id (e.g. "Condition").
standardOnly	If TRUE (default), return only standard concepts.
limit	Maximum rows to return. Default 100.

Value

A tibble with concept_id, concept_name, domain_id, vocabulary_id, concept_class_id, standard_concept, concept_code.

See Also

getConceptDescendants, mapSourceToStandard, getConceptSetDetails, cs

Examples

```
## Not run:
connection <- DatabaseConnector::connect(Eunomia::getEunomiaConnectionDetails())
searchConcepts("atrial fibrillation", connection, vocabularyDatabaseSchema = "main")
searchConcepts("metformin", connection, vocabularyDatabaseSchema = "main", domain = "Drug")

## End(Not run)
```

specimen	<i>Query the specimen domain</i>
----------	----------------------------------

Description

Query the specimen domain

Usage

```
specimen(conceptSet, ...)
```

Arguments

conceptSet	A specimen concept set
...	optional attributes (e.g. CorrelatedCriteria)

Value

A Capr Query

specimenAnatomicSite	<i>Specimen anatomic site attribute</i>
----------------------	---

Description

Specimen anatomic site attribute

Usage

```
specimenAnatomicSite(ids, connection = NULL, vocabularyDatabaseSchema = NULL)
```

Arguments

ids	integer concept ids for the attribute
connection	an optional connection to an OMOP CDM database, used only to look up concept names for display (e.g. in Atlas). When NULL (default) the attribute is built from the ids alone, which produces identical SQL.
vocabularyDatabaseSchema	the schema of the OMOP vocabulary tables; only used when 'connection' is supplied

Value

An attribute for use in specimen().

specimenAnatomicSiteCS

Specimen anatomic site filter via concept set

Description

Specimen anatomic site filter via concept set

Usage

```
specimenAnatomicSiteCS(conceptSet, isExclusion = FALSE)
```

Arguments

conceptSet	A ConceptSet built with cs().
isExclusion	logical; if TRUE the types in the concept set are excluded.

Value

An attribute for use in specimen().

specimenDiseaseStatus *Specimen disease status attribute*

Description

Specimen disease status attribute

Usage

```
specimenDiseaseStatus(ids, connection = NULL, vocabularyDatabaseSchema = NULL)
```

Arguments

ids	integer concept ids for the attribute
connection	an optional connection to an OMOP CDM database, used only to look up concept names for display (e.g. in Atlas). When NULL (default) the attribute is built from the ids alone, which produces identical SQL.
vocabularyDatabaseSchema	the schema of the OMOP vocabulary tables; only used when 'connection' is supplied

Value

An attribute for use in specimen().

`specimenDiseaseStatusCS`*Specimen disease status filter via concept set*

Description

Specimen disease status filter via concept set

Usage

```
specimenDiseaseStatusCS(conceptSet, isExclusion = FALSE)
```

Arguments

`conceptSet` A `ConceptSet` built with `cs()`.

`isExclusion` logical; if TRUE the types in the concept set are excluded.

Value

An attribute for use in `specimen()`.

`specimenSourceId`*Filter by specimen source ID text — Specimen*

Description

Filter by specimen source ID text — Specimen

Usage

```
specimenSourceId(op)
```

Arguments

`op` an `opAttributeCharacter` built with e.g. `stringContains("src")`.

Value

An attribute for use in `specimen()`.

specimenType	<i>Add a specimen type attribute to determine the provenance of the record</i>
--------------	--

Description

Add a specimen type attribute to determine the provenance of the record

Usage

```
specimenType(ids, connection = NULL, vocabularyDatabaseSchema = NULL)
```

Arguments

ids	integer concept ids for the attribute
connection	an optional connection to an OMOP CDM database, used only to look up concept names for display (e.g. in Atlas). When NULL (default) the attribute is built from the ids alone, which produces identical SQL.
vocabularyDatabaseSchema	the schema of the OMOP vocabulary tables; only used when 'connection' is supplied

Value

An attribute for use in specimen().

specimenTypeCS	<i>SpecimenType filter via concept set (ConceptSetSelection)</i>
----------------	--

Description

SpecimenType filter via concept set (ConceptSetSelection)

Usage

```
specimenTypeCS(conceptSet, isExclusion = FALSE)
```

Arguments

conceptSet	A ConceptSet built with cs().
isExclusion	logical; if TRUE the types in the concept set are excluded.

Value

An attribute for use in specimen().

specimenTypeExclude	<i>SpecimenTypeExclude query attribute</i>
---------------------	--

Description

When FALSE, specimen type is not excluded (include all). When TRUE, excludes specified specimen types (Capr does not support type-concept lists without DB).

Usage

```
specimenTypeExclude(exclude = FALSE)
```

Arguments

exclude	logical; FALSE = include all types (default), TRUE = exclude (unsupported in decompiler)
---------	--

Value

An attribute for use in specimen()

sponsorConcept	<i>Sponsor concept attribute for PayerPlanPeriod</i>
----------------	--

Description

Sponsor concept attribute for PayerPlanPeriod

Usage

```
sponsorConcept(conceptSet)
```

Arguments

conceptSet	A ConceptSet for the sponsor concept filter.
------------	--

Value

An attribute for use in payerPlanPeriod().

sponsorSourceConcept	<i>Sponsor source concept attribute for PayerPlanPeriod</i>
----------------------	---

Description

Sponsor source concept attribute for PayerPlanPeriod

Usage

sponsorSourceConcept(conceptSet)

Arguments

conceptSet A ConceptSet for the sponsor source concept filter.

Value

An attribute for use in payerPlanPeriod().

startDate	<i>Function that creates a start date attribute</i>
-----------	---

Description

Function that creates a start date attribute

Usage

startDate(op, type = "occurrence")

Arguments

op	an opAttribute object must be a date that defines the logical operation used to determine eligible start dates
type	specify the type of date to use either occurrence or era. default as occurrence

Value

An attribute that can be used in a cohort definition

stopReason	<i>Filter by stop reason text — ConditionOccurrence or DrugExposure</i>
------------	---

Description

Filter by stop reason text — ConditionOccurrence or DrugExposure

Usage

```
stopReason(op)
```

Arguments

op an opAttributeCharacter built with e.g. stringContains("reason").

Value

An attribute for use in conditionOccurrence() or drugExposure().

stopReasonConcept	<i>Stop reason concept attribute for PayerPlanPeriod</i>
-------------------	--

Description

Stop reason concept attribute for PayerPlanPeriod

Usage

```
stopReasonConcept(conceptSet)
```

Arguments

conceptSet A ConceptSet for the stop reason concept filter.

Value

An attribute for use in payerPlanPeriod().

stopReasonSourceConcept

Stop reason source concept attribute for PayerPlanPeriod

Description

Stop reason source concept attribute for PayerPlanPeriod

Usage

stopReasonSourceConcept(conceptSet)

Arguments

conceptSet A ConceptSet for the stop reason source concept filter.

Value

An attribute for use in payerPlanPeriod().

stringContains

String contains operator

Description

Builds a text-match operator for use with text filter attribute functions such as [lotNumber](#), [stopReason](#), [uniqueDeviceId](#).

Usage

stringContains(x)

S4 method for signature 'character'
stringContains(x)

Arguments

x Character substring to match.

Value

An opAttributeCharacter for use inside text attribute functions.

See Also

[stringStartsWith](#), [stringEndsWith](#), [stringNotContains](#), [lotNumber](#), [stopReason](#)

Examples

```
# Drug exposures where lot number contains "LOT"
drugExposure(cs(1L, name = "drug"), lotNumber(stringContains("LOT")))
```

stringEndsWith	<i>String ends with operator</i>
----------------	----------------------------------

Description

String ends with operator

Usage

```
stringEndsWith(x)
```

```
## S4 method for signature 'character'
stringEndsWith(x)
```

Arguments

x	Character suffix to match.
---	----------------------------

Value

An opAttributeCharacter for use inside text attribute functions.

See Also

[stringContains](#), [stringStartsWith](#), [lotNumber](#)

stringNotContains	<i>String does not contain operator</i>
-------------------	---

Description

String does not contain operator

Usage

```
stringNotContains(x)
```

```
## S4 method for signature 'character'
stringNotContains(x)
```

Arguments

x	Character substring to exclude.
---	---------------------------------

Value

An opAttributeCharacter for use inside text attribute functions.

See Also

[stringNotStartsWith](#), [stringNotEndsWith](#)

stringNotEndsWith	<i>String does not end with operator</i>
-------------------	--

Description

String does not end with operator

Usage

```
stringNotEndsWith(x)
```

```
## S4 method for signature 'character'  
stringNotEndsWith(x)
```

Arguments

x	Character suffix to exclude.
---	------------------------------

Value

An opAttributeCharacter for use inside text attribute functions.

See Also

[stringNotStartsWith](#), [stringNotContains](#)

stringNotStartsWith	<i>String does not start with operator</i>
---------------------	--

Description

String does not start with operator

Usage

```
stringNotStartsWith(x)
```

```
## S4 method for signature 'character'  
stringNotStartsWith(x)
```

Arguments

x	Character prefix to exclude.
---	------------------------------

Value

An opAttributeCharacter for use inside text attribute functions.

See Also

[stringNotContains](#), [stringNotEndsWith](#)

stringStartsWith	<i>String starts with operator</i>
------------------	------------------------------------

Description

String starts with operator

Usage

```
stringStartsWith(x)
```

```
## S4 method for signature 'character'
stringStartsWith(x)
```

Arguments

x	Character prefix to match.
---	----------------------------

Value

An opAttributeCharacter for use inside text attribute functions.

See Also

[stringContains](#), [stringEndsWith](#), [lotNumber](#)

summary	<i>Summarise a Capr object</i>
---------	--------------------------------

Description

Prints a fuller, nested view of a Capr S4 object than `show()` does. Every Capr summary method prints to the console and returns the object invisibly.

Usage

```
summary(object, ...)
```

Arguments

object	A Capr object (e.g. Cohort, CohortEntry, Query, Criteria, Group).
...	Additional arguments (currently unused).

Value

The object, invisibly.

toCirce	<i>Function to coerce cohort to circe</i>
---------	---

Description

Function to coerce cohort to circe

Usage

```
toCirce(cd)
```

Arguments

cd	the Capr cohort class
----	-----------------------

Value

an s3 list representing the circe object to coerce to json

toCohortJson	<i>Convert a Capr cohort to JSON</i>
--------------	--------------------------------------

Description

Convert a Capr cohort to JSON

Usage

```
toCohortJson(object, ..., includeConceptSets = NULL)
```

Arguments

object	A Capr cohort.
...	Arguments passed on to jsonlite::toJSON (e.g. pretty = TRUE).
includeConceptSets	Optional list of ConceptSet objects to include in the JSON even if not referenced in the cohort (e.g. for round-trip equivalence).

Value

A JSON character string for the cohort.

toConceptSetJson	<i>Convert a Capr concept set to JSON</i>
------------------	---

Description

Convert a Capr concept set to JSON

Usage

toConceptSetJson(object, ...)

Arguments

- object A Capr concept set created with cs().
- ... Arguments passed on to jsonlite::toJSON.

Value

A JSON character string for the concept set expression.

uniqueDeviceId	<i>Filter by unique device ID text — DeviceExposure</i>
----------------	---

Description

Filter by unique device ID text — DeviceExposure

Usage

uniqueDeviceId(op)

Arguments

- op an opAttributeCharacter built with e.g. stringContains("DV").

Value

An attribute for use in deviceExposure().

unitCS	<i>Unit filter via concept set — Measurement, Observation, DoseEra, Specimen</i>
--------	--

Description

Unit filter via concept set — Measurement, Observation, DoseEra, Specimen

Usage

```
unitCS(conceptSet, isExclusion = FALSE)
```

Arguments

conceptSet	A ConceptSet built with cs().
isExclusion	logical; if TRUE the types in the concept set are excluded.

Value

An attribute for use in measurement(), observation(), doseEra(), specimen().

valueAsConcept	<i>Add a value as concept attribute</i>
----------------	---

Description

Add a value as concept attribute

Usage

```
valueAsConcept(ids, connection = NULL, vocabularyDatabaseSchema = NULL)
```

Arguments

ids	integer concept ids for the attribute
connection	an optional connection to an OMOP CDM database, used only to look up concept names for display (e.g. in Atlas). When NULL (default) the attribute is built from the ids alone, which produces identical SQL.
vocabularyDatabaseSchema	the schema of the OMOP vocabulary tables; only used when ‘connection’ is supplied

Value

An attribute that can be used in a query function

valueAsConceptSet	<i>ValueAsConcept attribute from a concept set (for round-trip without DB)</i>
-------------------	--

Description

Filter Measurement or Observation by value_as_concept_id using a concept set. Use this when building from JSON (CodesetId reference); use valueAsConcept(ids) for ad-hoc concept ids.

Usage

valueAsConceptSet(conceptSet)

Arguments

conceptSet A ConceptSet object (e.g. from cs() or decompiled JSON).

Value

An attribute for use in measurement() or observation().

valueAsNumber	<i>Function to create valueAsNumber attribute</i>
---------------	---

Description

This function is used only for measurement query. valueAsNumber is a column in the measurement table of the cdm. This attribute allows a subquery to find measurements that satisfy certain values determined by the op logic.

Usage

valueAsNumber(op)

Arguments

op an opAttribute object that is either numeric or integer that defines the logical operation used to determine eligible patient age

Value

An attribute that can be used in a cohort definition

valueAsString	<i>Attribute for Observation value_as_string filter (round-trip from Atlas ValueAsString)</i>
---------------	---

Description

Serializes to ValueAsString: { Text, Op }; Circe generates WHERE value_as_string LIKE

Usage

valueAsString(text, op = "contains")

Arguments

- | | |
|------|--|
| text | Character string to match (e.g. "Yes"). |
| op | Circe op: "contains" (default, LIKE %text%), "starts", "ends", "equals". |

Value

Attribute for use in observation().

visit	<i>Query the visit occurrence domain</i>
-------	--

Description

Query the visit occurrence domain

Usage

visit(conceptSet, ...)

Arguments

- | | |
|------------|--|
| conceptSet | A ConceptSet built with cs() , or NULL for any visit. |
| ... | Optional attributes: firstOccurrence(), startDate(), endDate(), visitType(), visitLength(), visitSourceConcept(), placeOfService(), age(), nestedWithAll(), etc. |

Value

A Query object.

See Also

[entry](#), [cs](#)

Examples

```
ip <- cs(descendants(9201L, 262L), name = "Inpatient")
visit(ip)
```

visitDetail	<i>Query the visit detail domain</i>
-------------	--------------------------------------

Description

Query the visit detail domain

Usage

visitDetail(conceptSet, ...)

Arguments

- | | |
|------------|---|
| conceptSet | A visit detail concept set |
| ... | optional attributes (e.g. VisitDetailSourceConcept) |

Value

A Capr Query

visitDetailSourceConcept	<i>Add a visit detail source concept attribute</i>
--------------------------	--

Description

Add a visit detail source concept attribute

Usage

visitDetailSourceConcept(conceptSet)

Arguments

- | | |
|------------|--|
| conceptSet | a ConceptSet object containing the source concepts |
|------------|--|

Value

An attribute for use in visitDetail()

visitDetailTypeCS	<i>Visit detail type filter via concept set — VisitDetail</i>
-------------------	---

Description

Visit detail type filter via concept set — VisitDetail

Usage

```
visitDetailTypeCS(conceptSet, isExclusion = FALSE)
```

Arguments

conceptSet	A ConceptSet built with <code>cs()</code> .
isExclusion	logical; if TRUE the types in the concept set are excluded.

Value

An attribute for use in `visitDetail()`.

visitLength	<i>Visit length attribute for VisitOccurrence</i>
-------------	---

Description

Visit length attribute for VisitOccurrence

Usage

```
visitLength(op)
```

Arguments

op	an opAttribute object (numeric or integer) defining the length comparison in days
----	---

Value

An attribute for use in `visit()`.

visitSourceConcept	<i>Add a visit source concept attribute</i>
--------------------	---

Description

Add a visit source concept attribute

Usage

```
visitSourceConcept(conceptSet)
```

Arguments

conceptSet	a ConceptSet object containing the source concepts
------------	--

Value

An attribute that can be used in a query function

visitType	<i>Add a visit type attribute to determine the provenance of the record</i>
-----------	---

Description

Add a visit type attribute to determine the provenance of the record

Usage

```
visitType(ids, connection = NULL, vocabularyDatabaseSchema = NULL)
```

Arguments

ids	integer concept ids for the attribute
connection	an optional connection to an OMOP CDM database, used only to look up concept names for display (e.g. in Atlas). When NULL (default) the attribute is built from the ids alone, which produces identical SQL.
vocabularyDatabaseSchema	the schema of the OMOP vocabulary tables; only used when 'connection' is supplied

Value

An attribute for use in any domain query that has a visit context filter.

visitTypeCS	<i>VisitType filter via concept set (ConceptSetSelection)</i>
-------------	---

Description

VisitType filter via concept set (ConceptSetSelection)

Usage

visitTypeCS(conceptSet, isExclusion = FALSE)

Arguments

- conceptSet A ConceptSet built with cs().
- isExclusion logical; if TRUE the types in the concept set are excluded.

Value

An attribute for use in visit().

visitTypeExclude	<i>VisitTypeExclude query attribute</i>
------------------	---

Description

VisitTypeExclude query attribute

Usage

visitTypeExclude(exclude = FALSE)

Arguments

- exclude logical; FALSE = include all types (default), TRUE = exclude

Value

An attribute for use in visit()

visitTypeSet	<i>Add a visit type attribute (filter by visit_concept_id) from a concept set</i>
--------------	---

Description

Restricts criteria to events occurring in specified visit types (e.g. inpatient, ER). Used when decom-
piling cohort JSON that has VisitType as concept list or CodesetId.

Usage

```
visitTypeSet(conceptSet)
```

Arguments

conceptSet	a ConceptSet object containing visit type concepts
------------	--

Value

An attribute for use in conditionOccurrence(), drugExposure(), etc.

withAll	<i>Function to construct a group where all criteria and groups must be satisfied</i>
---------	--

Description

Function to construct a group where all criteria and groups must be satisfied

Usage

```
withAll(...)
```

Arguments

...	a set of criteria or groups
-----	-----------------------------

Value

an S4 group class specifying a bundle of criteria that all must be satisfied in context of the cohort
definition

withAny	<i>Function to construct a group where any criteria and groups may be satisfied</i>
---------	---

Description

Function to construct a group where any criteria and groups may be satisfied

Usage

```
withAny(...)
```

Arguments

... a set of criteria or groups

Value

an S4 group class specifying a bundle of criteria that any may be satisfied in context of the cohort definition

withAtLeast	<i>Function to construct a group where at least some of the criteria or groups must be satisfied</i>
-------------	--

Description

Function to construct a group where at least some of the criteria or groups must be satisfied

Usage

```
withAtLeast(x, ...)
```

Arguments

x an integer specifying the number of criteria or groups that must be satisfied
... a set of criteria or groups

Value

an S4 group class specifying a bundle of criteria that at least x be satisfied in context of the cohort definition

withAtMost	<i>Function to construct a group where at most some of the criteria or groups must be satisfied</i>
------------	---

Description

Function to construct a group where at most some of the criteria or groups must be satisfied

Usage

```
withAtMost(x, ...)
```

Arguments

x	an integer specifying the number of criteria or groups that must be satisfied
...	a set of criteria or groups

Value

an S4 group class specifying a bundle of criteria that at most x be satisfied in context of the cohort definition

writeCohort	<i>Write Cohort json file</i>
-------------	-------------------------------

Description

Write Cohort json file

Usage

```
writeCohort(x, path)
```

Arguments

x	A Capr cohort
path	The name of the file to create

Value

Invisibly returns the path to the json file that was written

Examples

```
## Not run:
cs1 <- cs(descendants(exclude(436665),440383,442306,4175329), name = "concepts")
# optional step to fill in concept set details. Requires database connection.

cs1 <- getConceptSetDetails(cs1, con)

x <- cohort(conditionOccurrence(cs1))
writeCohort(x, "cohortDefinition.json")

## End(Not run)
```

writeConceptSet	<i>Save a concept set as a json file</i>
-----------------	--

Description

The resulting concept Set JSON file can be imported into Atlas.

Usage

```
writeConceptSet(x, path, format = "auto", ...)
```

Arguments

x	A Capr concept set created by 'cs()'
path	Name of file to write to. (e.g. "concepts.json")
format	the file extension to write
...	additional arguments

Examples

```
anemia <- cs(descendants(439777,4013073,4013074), name = "anemia")
writeConceptSet(anemia, file.path(tempdir(), 'anemia.json'))
writeConceptSet(anemia, file.path(tempdir(), 'anemia.csv'))
```

Index

`==`, 6
`==`, `ConceptSet`, `ConceptSet`-method (`==`), 6

`age`, 7, 11, 12, 38, 50, 51, 55
`ageAtEnd`, 7, 7
`ageAtStart`, 7, 8
`as.data.frame`, `ConceptSet`-method, 8
`as.json`, 9
`as.json`, `Cohort`-method (`as.json`), 9
`as.json`, `ConceptSet`-method (`as.json`), 9
`atLeast`, 9, 10, 17, 43
`atMost`, 10, 10, 43
`attrition`, 11, 13, 38

`bt`, 7, 11, 39, 51, 55, 56, 63
`bt`, `Date`-method (`bt`), 11
`bt`, `integer`-method (`bt`), 11
`bt`, `numeric`-method (`bt`), 11

`CensoringCriteria`-class, 12
`censoringEvents`, 13, 25, 44
`Cohort`, 49
`cohort`, 11, 13, 38, 40, 44, 53
`compile`, 14
`compile`, `Cohort`-method, 14
`compile`, `ConceptSet`-method
 (`compile`, `Cohort`-method), 14
`Concept`-class, 15
`conceptAttribute`-class, 15
`ConceptSet`-class, 16
`conceptSetAttribute`-class, 16
`ConceptSetItem`-class, 16
`conditionEra`, 17, 71
`conditionOccurrence`, 17, 17
`conditionSourceConcept`, 18
`conditionStatus`, 18
`conditionStatusCS`, 19
`conditionType`, 19
`conditionTypeCS`, 20
`conditionTypeExclude`, 20
`continuousObservation`, 21, 38
`Criteria`-class, 21
`cs`, 17, 22, 27, 32, 53, 59, 65, 77, 85, 103

`dateAdjustment`, 23

`dateAdjustmentAttribute`-class, 23
`daysOfSupply`, 24
`death`, 24
`deathType`, 25
`deathTypeCS`, 25
`deathTypeExclude`, 26
`descendants (cs)`, 22
`detectUnsupportedKeys`, 26
`deviceExposure`, 27
`deviceType`, 27
`deviceTypeCS`, 28
`deviceTypeExclude`, 28
`doseEra`, 29, 30
`doseUnit`, 29
`doseUnitCS`, 30
`doseValue`, 30
`drugEra`, 31, 32, 40
`drugExit`, 31, 31, 44
`drugExposure`, 31, 32, 54
`DrugExposureExit`-class, 33
`drugQuantity`, 33
`drugRefills`, 34
`drugSourceConcept`, 34
`drugType`, 35
`drugTypeCS`, 35
`drugTypeExclude`, 36
`duringInterval`, 9, 10, 36, 43

`endDate`, 37
`Endpoint`-class, 37
`entry`, 11, 13, 17, 27, 31, 32, 37, 59, 65, 77, 103

`eq`, 7, 38, 51, 55, 56
`eq`, `character`-method (`eq`), 38
`eq`, `Date`-method (`eq`), 38
`eq`, `integer`-method (`eq`), 38
`eq`, `numeric`-method (`eq`), 38
`era`, 13, 39
`eraLength`, 40
`ethnicityConcepts (male)`, 57
`ethnicityCS`, 40
`EventAperture`-class, 41
`eventEnds`, 41
`eventStarts`, 42

- EventWindow-class, 42
- exactly, 10, 43
- exclude (cs), 22
- exit, 13, 25, 38, 44
- female (male), 57
- firstOccurrence, 44
- FixedDurationExit-class, 45
- fixedExit, 44, 45
- genderConcepts, 46
- genderCS, 46
- getConceptDescendants, 47
- getConceptInfo, 48
- getConceptSetCall, 48
- getConceptSetDetails, 49
- getConceptSetDetails, Cohort-method (getConceptSetDetails), 49
- getConceptSetDetails, ConceptSet-method (getConceptSetDetails), 49
- Group-class, 50
- gt, 7, 12, 39, 50, 51, 55, 56, 63
- gt, Date-method (gt), 50
- gt, integer-method (gt), 50
- gt, numeric-method (gt), 50
- gte, 7, 39, 51, 51, 55, 56
- gte, Date-method (gte), 51
- gte, integer-method (gte), 51
- gte, numeric-method (gte), 51
- insertCaprCode, 52
- jsonToCapr, 52, 52, 53
- jsonToCaprFile, 52, 53
- keyValueAttribute-class, 54
- logicAttribute-class, 54
- lotNumber, 54, 95, 96, 98
- lt, 7, 12, 39, 51, 55, 56, 63
- lt, Date-method (lt), 55
- lt, integer-method (lt), 55
- lt, numeric-method (lt), 55
- lte, 7, 39, 51, 55, 55
- lte, Date-method (lte), 55
- lte, integer-method (lte), 55
- lte, numeric-method (lte), 55
- makeCohortSet, 56
- male, 57
- mapped (cs), 22
- mapSourceToStandard, 58
- measurement, 59, 83, 102
- measurementOperator, 59
- measurementOperatorCS, 60
- measurementSourceConcept, 60
- measurementType, 61
- measurementTypeCS, 61
- measurementTypeExclude, 62
- measurementUnit, 59, 62
- nbt, 7, 12, 63
- nbt, Date-method (nbt), 63
- nbt, integer-method (nbt), 63
- nbt, numeric-method (nbt), 63
- nestedAttribute-class, 63
- nestedWithAll, 64
- nestedWithAny, 64
- nestedWithAtLeast, 64
- nestedWithAtMost, 65
- observation, 65, 102
- observationExit, 44, 66
- ObservationExit-class, 66
- observationPeriod, 66
- observationPeriodType, 67
- observationQualifier, 67
- observationQualifierCS, 68
- observationSourceConcept, 68
- observationType, 69
- observationTypeCS, 69
- observationTypeExclude, 70
- ObservationWindow-class, 70
- Occurrence-class, 70
- occurrenceCount, 71
- opAttributeCharacter-class, 71
- opAttributeDate-class, 71
- opAttributeInteger-class, 72
- opAttributeNumeric-class, 72
- opAttributeSuper-class, 72
- payerConcept, 73
- payerPlanPeriod, 73
- payerSourceConcept, 74
- periodLength, 74
- periodTypeCS, 75
- placeOfService, 75
- placeOfServiceCS, 76
- planConcept, 76
- planSourceConcept, 77
- procedure, 77
- procedureModifier, 78
- procedureModifierCS, 78
- procedureSourceConcept, 79
- procedureType, 79
- procedureTypeCS, 80
- procedureTypeExclude, 80

- providerSpecialtyConcepts, [81](#)
- providerSpecialtyCS, [81](#)
- quantityValue, [82](#)
- Query-class, [82](#)
- raceConcepts (male), [57](#)
- raceCS, [82](#)
- rangeHigh, [83](#)
- rangeHighRatio, [83](#)
- rangeLow, [84](#)
- rankedSearchConcepts, [84](#)
- readConceptSet, [85](#)
- routeConcept, [86](#)
- routeConceptCS, [86](#)
- searchConcepts, [85](#), [87](#)
- specimen, [88](#)
- specimenAnatomicSite, [88](#)
- specimenAnatomicSiteCS, [89](#)
- specimenDiseaseStatus, [89](#)
- specimenDiseaseStatusCS, [90](#)
- specimenSourceId, [90](#)
- specimenType, [91](#)
- specimenTypeCS, [91](#)
- specimenTypeExclude, [92](#)
- sponsorConcept, [92](#)
- sponsorSourceConcept, [93](#)
- startDate, [11](#), [38](#), [50](#), [51](#), [55](#), [93](#)
- stopReason, [94](#), [95](#)
- stopReasonConcept, [94](#)
- stopReasonSourceConcept, [95](#)
- stringContains, [95](#), [96](#), [98](#)
- stringContains, character-method
(stringContains), [95](#)
- stringEndsWith, [95](#), [96](#), [98](#)
- stringEndsWith, character-method
(stringEndsWith), [96](#)
- stringNotContains, [95](#), [96](#), [97](#)
- stringNotContains, character-method
(stringNotContains), [96](#)
- stringNotEndsWith, [96](#), [97](#), [97](#)
- stringNotEndsWith, character-method
(stringNotEndsWith), [97](#)
- stringNotStartsWith, [96](#), [97](#), [97](#)
- stringNotStartsWith, character-method
(stringNotStartsWith), [97](#)
- stringStartsWith, [95](#), [96](#), [98](#)
- stringStartsWith, character-method
(stringStartsWith), [98](#)
- summary, [98](#)
- toCirce, [99](#)
- toCohortJson, [13](#), [15](#), [99](#)
- toConceptSetJson, [100](#)
- uniqueDeviceId, [95](#), [100](#)
- unitCS, [101](#)
- valueAsConcept, [101](#)
- valueAsConceptSet, [102](#)
- valueAsNumber, [11](#), [38](#), [50](#), [51](#), [55](#), [59](#), [102](#)
- valueAsString, [103](#)
- visit, [103](#)
- visitDetail, [104](#)
- visitDetailSourceConcept, [104](#)
- visitDetailTypeCS, [105](#)
- visitLength, [105](#)
- visitSourceConcept, [106](#)
- visitType, [106](#)
- visitTypeCS, [107](#)
- visitTypeExclude, [107](#)
- visitTypeSet, [108](#)
- withAll, [10](#), [11](#), [38](#), [43](#), [108](#)
- withAny, [10](#), [11](#), [43](#), [109](#)
- withAtLeast, [11](#), [109](#)
- withAtMost, [11](#), [110](#)
- writeCohort, [13](#), [110](#)
- writeConceptSet, [111](#)